SUP'COM

المـــدرســـة العليـــا للمـــواصـــلات بتونـــس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunication Engineering Studies

*Major: Management of innovative technologies (MIT)*

# Graduation Project Report

*Topic:*

# Conception of an IoT network for vineyard monitoring

*By:*

**Taha BEN SALAH**

*Supervised by:*

Mr Kim Khoa Nguyen
Mr Chiheb Rebai

*Project in collaboration with:*

Media@ Synchromedia

ÉTS Le génie pour l'industrie — ÉCOLE DE TECHNOLOGIE SUPÉRIEURE, Université du Québec

Academic year: 2017 - 1ed87414c290016su

Signature




*In the name of Allah*

*The most gracious, the most merciful*

# Dedication

**To the souls of my grandparents,**
The first who taught me to be who I am, the souls I still miss every day,

**To my lovely mother,**
For her love, kindness, and her support through my hardest moments,

**To my great father,**
For his sacrifice and devotion, his selflessness will be always remembered,

**To my dear brothers and sister,**
For their encouragements and affections,

**To my friends,**
For being there for me through good and bad moments,

**To all those who have known defeat and suffering, pain and struggle,**
And have found their way out of this hardship, motivating me
to always move on and never give up,

**To all those I have met,**
For helping - directly or indirectly - shaping the person who I am today,

I dedicate this modest work.

# Acknowledgement

I am very pleased to express my appreciation and gratitude to all those who have helped, assisted and guided me throughout this project, in a direct or indirect manner.

In particular, I would like to express my sincere gratitude to my supervisor at *Synchromedia Lab* (ÉTS), Professor Kim Khoa Nguyen, for accepting me as a member in his team, for providing me all the necessary materials, and for being available every time I needed him. Without his assistance, this project could not have been successful.

I am also very thankful to Ghazi Ben Amor, a master's student in the lab who was always available whenever I had a question about the project, to Sadegh Aghabozorgi and Meriem Mechennef, another two master's students, for their encouragements and lovely work atmosphere, and for their company during my work. I address, likewise, my thanks to all the rest of *Synchromedia Lab* members.

I would like to convey my special regards to my academic supervisor, Professor Chiheb Rebai, for his regular feedback, friendly assistance and enthusiastic encouragement.

May all the members of the Jury find also my gratitude for being those who will judge my modest work.

# Abstract

In this report, we discuss the importance of the internet of things technology (IoT) and its application in the agriculture. The impact of such technology in making daily life tasks an automatic routine, hence reducing costs for the client and allowing him gaining accuracy of measurements and reducing time consumption.

Based on these concepts, we describe in this project the need for such smart solutions to implement in farms, making farmers having better control over their fields and better monitoring overs their plants.

We are dealing with real client owning an important vineyard in France, and willing to implement such technical solution in his field in order to have better control over his grapevines, which will lead him at the end to produce with better quality while consuming fewer resources.

**Key Words:**

Internet of things, smart agriculture, LoRa communication, raspberry pi, embedded systems, electronics, sensor node.

# Table of contents

**Dedication** ..... i
**Acknowledgement** ..... ii
**Abstract** ..... iii
**General Introduction** ..... 1

**Chapter 1: General context** ..... 2
**1. Hosting University presentation** ..... 2
1.1. Mitacs ..... 2
1.2. ÉTS *(École de Technologie Supérieure)* ..... 2
1.3. Synchromedia Lab ..... 3
**2. Basic definitions** ..... 3
2.1. Smart Object ..... 3
2.2. The Internet of things (IoT) ..... 5
**3. How IoT can be used for agriculture** ..... 7
**4. Work perspective** ..... 9
**Conclusion** ..... 10

**Chapter 2: Specification and design** ..... 11
**1. Introducing the project** ..... 11
**2. Vineyard monitoring** ..... 11
2.1. Light ..... 12
2.2. Temperature ..... 12
2.3. Relative humidity ..... 12
2.4. Wind ..... 13
**3. Project objectives** ..... 13
**4. The sensor node design** ..... 15
4.1. Choosing the platform ..... 15
4.2. Choosing the operating system ..... 16
4.3. Choosing the video/image capturing technique ..... 17
**5. The Gateway design** ..... 18
**6. The communication protocol** ..... 19
**7. The flow chart** ..... 21
**Conclusion** ..... 23

**Chapter 3: Implementation and results** ..... **24**
**1. Working environment** ..... **24**
**2. Setting up the sensor node** ..... **25**
2.1. The DS18b20 sensor ..... 26
2.2. The SKU: SEN0193 sensor ..... 28
2.3. The DHT22 sensor ..... 30
2.4. The raspberry pi Camera module ..... 32
**3. Setting up the Gateway** ..... **33**
3.1. The LoRa module ..... 33
3.2. LoRa transmission test ..... 34
**4. Connecting the whole system** ..... **42**
**Conclusion** ..... **44**

**General conclusion** ..... **45**
**References** ..... **46**
**ANNEXE** ..... **48**

# List of figures


Figure 1: Mitacs Logo ..... 2
Figure 2: ÉTS logo ..... 2
Figure 3: Synchromedia lab logo ..... 3
Figure 4: Hardware architecture of a smart object ..... 4
Figure 5: Process of software development for a smart object ..... 4
Figure 6: Appearance of the IoT (between 2008 and 2009) ..... 6
Figure 7: projected world population until 2100 ..... 8
Figure 8: the Dew on a grapevine leaf ..... 12
Figure 9: the Mildew fungi on a grapevine leaf ..... 12
Figure 10: the evapotranspiration ..... 13
Figure 11: the Raspberry pi board: a size comparison with a quarter dollar ..... 16
Figure 12: CSI connector on the raspberry pi (on the left) ..... 17
Figure 13: The raspberry pi 3 ..... 19
Figure 14: Lora technology logo ..... 21
Figure 15: Flow chart of the system ..... 22
Figure 16: four wires (red, black, green and white) connected to GPIO pins ..... 25
Figure 17: WiringPi pins numbering ..... 26
Figure 18: DS18b20: a temperature sensor for the soil ..... 26
Figure 19: a DS18b20 sensor connected to the raspberry pi zero ..... 27
Figure 20: DS18b20 sensor output sample ..... 27
Figure 21: Soil moisture sensor ..... 28
Figure 22: Analogue to digital converter (the MCP3008) ..... 28
Figure 23: Soil moisture sensor connected through an ADC to the raspberry pi ..... 28
Figure 24: Different measurement regions for the soil humidity sensor calibration ..... 29
Figure 25: SKU:SEN0193 sensor output sample ..... 30
Figure 26: a DHT22 sensor ..... 30
Figure 27: the DHT22 sensor connected to the raspberry pi ..... 31
Figure 28: DHT22 sensor output sample ..... 31
Figure 29: A camera module v2.1 with a protective plastic shield is connected to the sensor node ..... 32
Figure 30: Mildew (to the left) and Black rot (to the right) ..... 32
Figure 31: an input image (to the left) VS the processed image (to the right) ..... 33
Figure 32: Different LoRa radio modules supported by raspberry pi ..... 33
Figure 33: the LoRa module connected to the raspberry pi 3 ..... 34
Figure 34: the Sensor node and Gateway configuration (1) ..... 34
Figure 35: MobaXterm software logo ..... 35
Figure 36: the Sensor node and Gateway configuration (2) ..... 35
Figure 37: Gateway running in standalone mode ..... 36
Figure 38: A ping-pong test between the sensor node and the gateway (sensor node perspective) ..... 36
Figure 39: A ping-pong test between the sensor node and the gateway (gateway perspective) ..... 37
Figure 40: Temperature transmission (sensor node perspective) ..... 38
Figure 41: MAC address of the sensor node used as the transmission ID ..... 39
Figure 42: Temperature transmission (gateway perspective) ..... 39
Figure 43: the Gateway web admin interface ..... 40
Figure 44: the Log file in the Gateway ..... 41
Figure 45: temperature received as shown in The Thing Speaks Dashboard ..... 41
Figure 46: The sensor node fully assembled ..... 42
Figure 47: Output sample of the final sensor node ..... 43
Figure 48: Breadboard view of the sensor node ..... 51
Figure 49: Breadboard view of the gateway ..... 51
Figure 50: Schematic view of the sensor node ..... 52
Figure 51: Schematic view of the Gateway ..... 53

# List of Tables

TABLE 1: EMBEDDED BOARDS COMPARISON ..... 15
TABLE 2: RASPBERRY PI MODELS COMPARISON ..... 18
TABLE 3: EXISTING WIRELESS SOLUTIONS OVERVIEW ..... 20

# General Introduction

Nowadays, with the scientific and technologic advancement we reach, all life aspects of the modern society are affected by technology. High-tech revolution affected every industry that we can no more imagine anything left without being the host, in a way or another, of a High-tech component. Everything around us is becoming a smart device on itself able to perform different tasks, from easy and simple ones to the most complicated and complex routine.

The number of smart devices and objects connected to the internet is impressively increasing during the last couple of years, providing an endless number of applications, ranging from monitoring machines to smart disaster management, smart cities or healthcare applications, which require massive data transfer.

The common vision of all these applications is associated with the *Internet of Things* (IoT): the technology which allows various objects to communicate between themselves in order to perform a given task. IoT can be applied in the agriculture field, called also *smart agriculture.*

The aim of this project is to implement a prototype of an IoT solution to be used by a client owning a vineyard. The project will satisfy the needs of our client in terms of requested criteria for monitoring his plants. We will implement a sensor node collecting different measures such as temperature, humidity and so on, and sending them to a gateway for processing.

This work has been carried out in the context of an exchange program called *Mitacs*, a program matching different international students with various Canadian universities. The project we developed at ÉTS (*École de Technologie Supérieure*, Montreal) is a mandatory project for my final year of studies, in order to obtain an engineering degree in Telecommunications from SUP'COM.

This report will be divided into 3 main chapters: The first one deals with the theory of the Internet of Things (IoT), the second one deals with the design and conception of our system, and the third one is about the implementation and results.

# Chapter 1: General context

This chapter mainly focuses on the general context of our project. We will briefly introduce the context of the exchange program and the hosting university. Then we will focus on basic definitions needed for this work as well as the general problematic.

## 1. Hosting University presentation

### 1.1. Mitacs

This work has been carried out as a financed project by Mitacs. It is a Canadian not-for-profit organization, having as a mission to build "*partnership between academia, industry, and the word, to create a more innovative Canada*".

Among the various programs proposed by Mitacs, this project has been carried out as a *Globalink Research Internship*, a program in which Mitacs matches the top-ranked applications from Australia, Brazil, China, France, Germany, India, Mexico, Saudi Arabia, Tunisia and Ukraine, with more than 45 universities across Canada.[1]

*Figure 1: Mitacs Logo*

### 1.2. ÉTS *(École de Technologie Supérieure)*

Being accepted as a *Mitacs Gloablink intern*, I have been matched with ÉTS to carry out this project.

ÉTS is an engineering school in Montreal, belonging to the University of Quebec and known for its excellent quality in research and innovation. It is also known for its unique partnership with the industry, ranging from large to medium and small companies. ÉTS figures on top 5 engineering schools in Canada.[2]

Le génie pour l'industrie

*Figure 2: ÉTS logo*

[1] Mitacs web site, Globalink Research Internship
[2] ETS web site « *l'ÉTS en bref / Présentation* »

### 1.3. Synchromedia Lab

This Project has been carried out inside the Synchromedia Lab at ÉTS. This research group was created in 1988 and belongs to the Automation Engineering Department. The lab works on various projects related to many pertinent research areas such as machine learning, computer vision, internet of things, 5G, cloud computing, optic networks and so on. It consists of many renown professors and researchers, as well as adjunct members from other departments and universities.[3]




*Figure 3: Synchromedia lab logo*

## 2. Basic definitions

### 2.1. Smart Object

An object is called “smart”, according to his physical appearance (what we call the hardware), and at the same time by his behaviour (called the software).

#### *2.1.1. The Hardware*

The hardware design of an object in order to make it “smart object” contains electrical circuits typically consisting of four main components:

- Sensors and actuators :

The sensors and actuators give the device the ability to interact with its surrounding environment. Sensors allow sensing specific criteria such as temperature, pressure, humidity etc. while actuators allow acting upon the environment and affect it. Sensors and actuators ensure the interaction with the physical world.

- Communication device :

The communication device is the component which makes the smart object able to communicate. It can be either a wire in case of wired communication such as an Ethernet connection, or a radio transceiver in case of wireless communication. The communication device gives the smart object the ability to function both as a transmitter and a receiver.

- Microcontroller :

The microcontroller gives the smart object its “behaviour”. It runs the software needed for the function of the device and plays a central role in connecting the sensors with the communication device. It is the microcontroller that gives the smart object its intelligence.

[3] Synchromedia Lab web site

- Power source :

A power source is required since the smart object is made out of electrical circuits. It can be a battery, solar energy, or any other power source.

Based on whether the communication device is for wireless or wired communication, we can represent the hardware architecture of a smart object with one of the following two diagrams: [4]

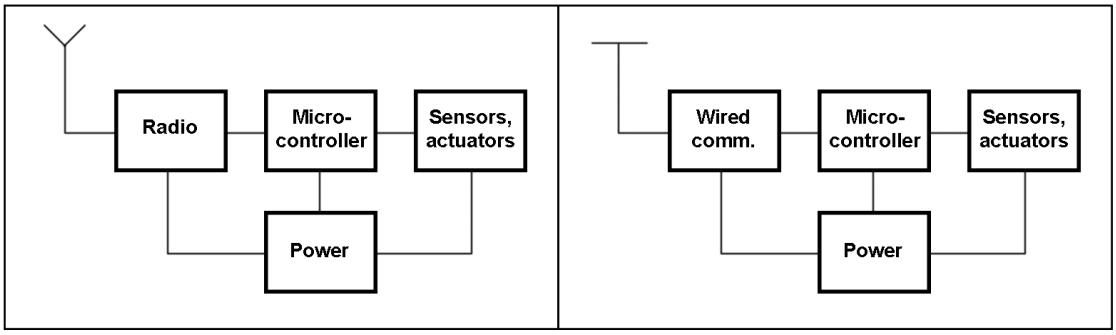



*Figure 4: Hardware architecture of a smart object*

### 2.1.2. The software

Being run by the microcontroller, it is the software that will define the behaviour of the smart object. It can be written by various programming languages such as C or C++, then compiled to a machine code to be written to the microcontroller.

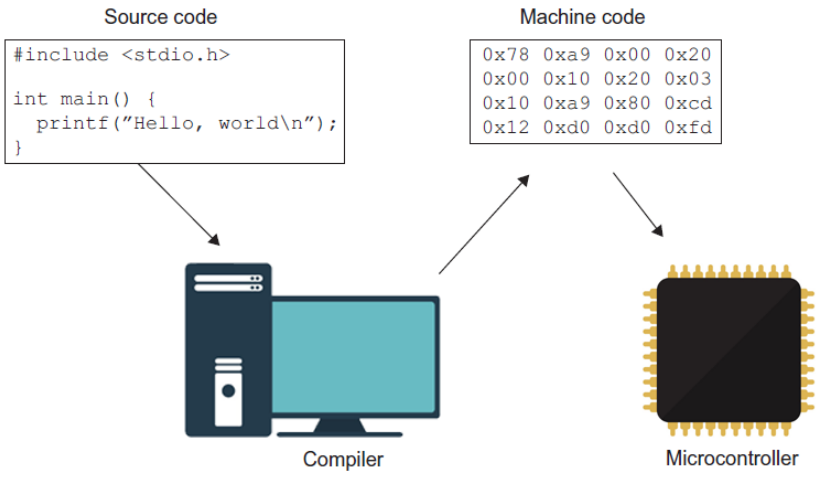



*Figure 5: Process of software development for a smart object* [5]

A smart device has its own operating system (OS). A large-scale OS like Linux, Mac, or Windows used basically in general purpose computers, are impossible to implement in such device due to its hardware severe constraints (weaker processing power, significantly less memory, etc.).

The communication protocol used by the smart device will be implemented by its software.

[4] Interconnecting smart objects with IP : the next internet (Vasseur and Dunkels, 2010)
[5] Interconnecting smart objects with IP : the next internet (Vasseur and Dunkels, 2010)

Having multiple smart devices connected together in the same network will make what we call *the Internet of Things*.

### 2.2. The Internet of things (IoT)

According to the ITU (International Telecommunications Union), the internet of things is:

"*a global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on, existing and evolving, interoperable information and communication technologies".* [6]

In a technical report made by Cisco over the IoT technology published in June 2014, they define it as follows:

*"The internet of things is the network of physical objects accessed through the internet, as defined by technology analysts and visionaries. These objects contain embedded technology to interact with internal states or the exterior environment".* [7]

In other words, IoT can be seen as an extension for the internet from the electric world to the things in our physical world: Internet on its classic meaning does not go beyond the electric cables, while the internet of things goes beyond this limit to take in consideration various objects from our real world, and forward its communications through the internet network.

You can have a smart fridge, a smart lamp, a smart oven, a smart desk, a smartwatch, a smartphone, and many other objects around you, connected to each other and sending data from one device to another, and this *"network of things"* is connected to the internet.

The internet of things is transforming our society, including ourselves. It allows improving considerably the capacity of the internet to collect, to analyze, and to restitute data that we transfer - in a second stage – to information and to knowledge.

In 2003, the world population was about 6.3 billion people while only 500 million devices were connected to the internet. [8]

Taking in consideration these statistics, if we divide the total number of devices over the global world population, we get less than one (0.08) connected device per person, which means technically, the IoT did not exist before 2003 since the number of connected devices is relatively very low.

The most popular connected device nowadays is the smartphone, and back to 2003, these smartphones were just about to start. The iPhone, for example, was announced only on January 9, 2007, by Steve Jobs the CEO and co-founder of Apple, during the Macworld conference. [9]

The massive use of smartphones and tablet made the number of connected devices in 2010 reach 12.5 billion, while the global population were 6.8 billion people.

---

[6] Intro to Internet of Things, Marco Zennaro, PhD, December 2016
[7] Intro to Internet of Things, Marco Zennaro, PhD, December 2016
[8] *« Comment l'évolution actuelle d'Internet transforme-t-elle le monde ? »* by Dave Evans, April 2011
[9] *« Comment l'évolution actuelle d'Internet transforme-t-elle le monde ? »* by Dave Evans, April 2011

It was at that time when the number of connected devices by person exceeded one (1.84) for the first time in history. [10]

Cisco adjusted further these numbers and considered the appearance of the IoT was between 2008 and 2009. [11]

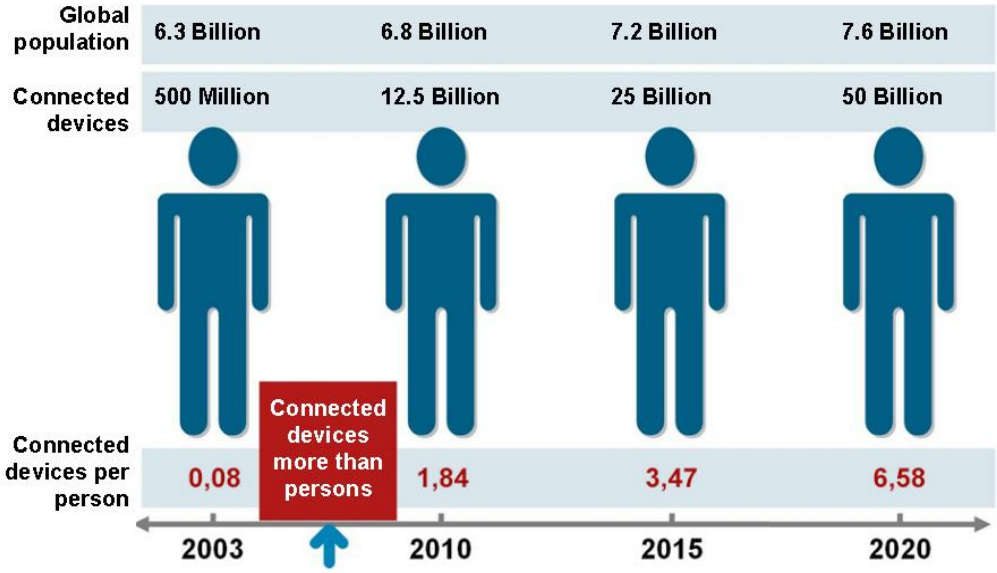



*Figure 6: Appearance of the IoT (between 2008 and 2009)*

The IoT can be applied in different fields:

- Smart grid :

The power grid is the electrical network delivering electricity to different consumers ranging from individuals in houses to large offices and industrial buildings. Electricity is produced and then transported to the end user after different stages of voltage transformation.

The Smart grid initiative involves a whole transformation of the classic network to involve billions of smart devices connected together in order to ensure better power management, to reduce carbon emission (due to fossil fuel consumption), to support new sources of energy (mainly renewable and green energy), to reduce cost and achieve a higher level of reliability.

- Industrial Automation :

It refers to making the industrial process and automatic task thanks to the use of modern computer-assisted technology and smart devices. Available access to hazardous and dangerous places inside the cite and the easy installation due to the wireless communication is among the main reasons why IoT is used in such applications. It makes diagnostic and maintenance faster, safer, and more effective

[10] *« Comment l'évolution actuelle d'Internet transforme-t-elle le monde ? »* by Dave Evans, April 2011
[11] *« Comment l'évolution actuelle d'Internet transforme-t-elle le monde ? »* by Dave Evans, April 2011

through wireless communication compared to the traditional wired industrial automation.

- Smart Cities and Urban Networks :

The integration of IoT with development projects is changing the landscape of modern cities. Transport officials use smart systems to reduce pollution and traffic congestion. Security officials are more apt to solve incidents and respond faster to emergencies, they are even able to predict crimes which help to prevent them before it occurs.

In modern cities, IoT networks manage the outdoor lightening system more efficiently, thus significantly reduce energy consumption. Vehicle tracking and parking management are improved with IoT. Smart cities improve the overall life quality of citizens.

- Home automation :

IoT provides solutions that help individuals easily control and manage their houses. Diverse applications are included such as lighting control, water consumption control, access control, remote control, comfort and convenience, kids and elderly assistance, etc.

- Building automation :

The previous concept of house automation can be extended to fit large scale buildings with additional applications. Smart devices are spread all around the building for better control on air conditioning and ventilation, intelligent fire and evacuation control, intelligent energy management, etc.

- Container tracking :

According to the International Chamber of Shipping, 90% of all goods in the global trade are shipped in containers. [12]

The ability to track these containers, know about its precise locations, its status, and remotely inspect its conditions for both the shipping company and its client, is very important for the global trade. Smart objects in the IoT make such task more reliable and more efficient.

- Smart agriculture :

IoT can be also used for agriculture to improve many aspects. Since our project is a case of IoT application in agriculture, we will develop further this aspect in the next section.

## 3. How IoT can be used for agriculture

To understand the importance of improving efficiency in the agricultural field, we need to start first by having an idea how much important this should be for the future generations.

[12] International Chamber of Shipping /Shipping and world trade, 2017

Each second, we have approximately 3 more persons in the global population. This means an additional 240 000 persons a day according to the Food and Agriculture Organization (FAO). [13]

In June 2017, the Department of Economic and Social Affairs of the United Nations published a revision in which they expect the global population to reach 8.6 Billion in 2030, and 9.8 Billion in 2050: [14]

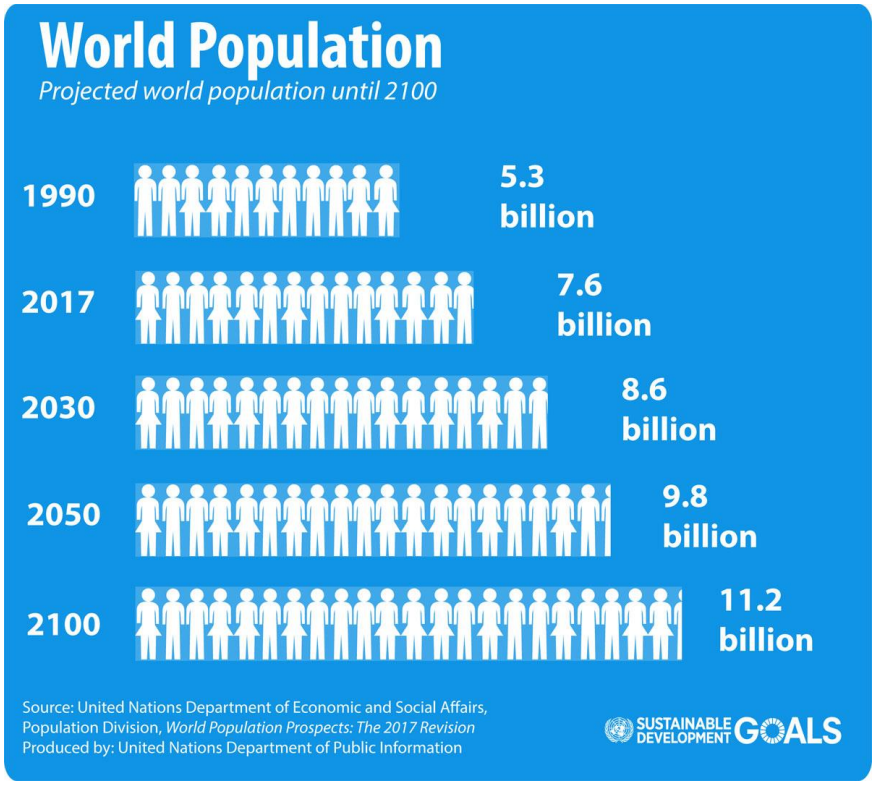



*Figure 7: projected world population until 2100*

Within the next decade, an additional billion people on the planet need to be fed. Adding to this the climate change which affects the global production negatively, having a sustainable food security for the next generation becomes a huge challenge.

The global demand for food is expected to grow by 70% in the first half of this century[15]. The Secretary of CODEX Alimentarius Commission (responsible for food quality and safety standards) Mr Tom Heilandt Said: *"Agriculture exists first to feed people and has done so for thousands of years, and it will have to continue doing the same for hopefully many more".* [16]

This implies farmers have to increase significantly their actual production for food security. While sunshine and air needed for photosynthesis are costless resources, other farmers' requirements such as fuel, fertilizers, sprays, pesticides and so on, are a heavy investment. Water is also an expensive resource in both

[13] Smart Farming, ISO focus #122, Mai-June 2017
[14] UN news "World population to hit 9.8 billion by 2050 despite nearly universal lower fertility rates" - June 2017
[15] How to feed the world in 2050, FAO
[16] Smart Farming, ISO focus #122, Mai-June 2017

environmental and financial aspects. The World health organization (WHO) expect that half of the world's population will be living in water-stressed area by 2025. [17]

This has been said, farmers are facing a huge challenge: they need to increase their production for the global growing population, with a better resource usage, and at the same time when climate change is negatively impacting the environment. But they cannot do it alone and they cannot do it with the actual farming practices. Luckily for them, technology is always here for help.

Introducing the IoT in agriculture makes farming more efficient by ensuring the crops and soil will receive exactly the needed quantity of water and other materials, for a better productivity, health, and resources saving. Here's where precise farming comes in thanks to the IoT, the technology that pushes farming to the next level.

With IoT in agriculture, we can have:

- Smart Irrigation.
- Weather monitoring and forecasting.
- Remote monitoring of crop.
- Remote monitoring of soil quality.
- Smart warehousing, logistics and distribution.
- Etc.

## 4. Work perspective

To sum up what has been said in the previous section in a simpler manner, we are facing a fast-growing population that requires to be fed, under two severe constraints: climatic changes that impact negatively the agriculture and water scarcity expected for future generations. We need a solution to ensure better productivity with fewer resources consumption.

In this perspective, the work we have done during this project is an intelligent solution for agriculture:

A solution that provides precise monitoring of atmospheric parameters required for the growth of the plant, which will ensure controlling the plant development and the fruit quality. Fruit quality is a key factor for feeding the next generations enough nutrients.

We propose a monitoring system for health control over the crops in order to early detect any risk, ensuring in this way saving the plant before it's too late for any intervention. This will prevent infections from spreading all over the field and plants from dying. Future farming will be more sensitive, and every plant matters! We cannot tolerate any risk of plant death for food security.

Our solution will ensure accurate control over soil humidity. Knowing the exact water level in the soil will make irrigation systems more precise and optimal. Precise water management will be more important for future generations under the water scarcity constraint. Monitoring other soil parameters gives the farmer better

[17] World Health Organization, Drinking-water – February 2018

understanding over its quality, which will lead to an optimal usage of fertilizers, another resource that must be well managed in order to reduce resource consumption and face the future resources scarcity risk.

In this perspective, our project is an example of smart agriculture solution needed for one of the most critical future challenges: food security. More details about the work will be developed further in the rest of this report. Even though our solution has been implemented for a particular type of plants (grapevines), the same concept can be adjusted for any other crop.

## Conclusion

In this introductive chapter, we defined how to classify a device as being a smart object or not, and how connecting and integrating multiple smart objects together in a network with human interaction makes what we call the *Internet of Things* (IoT).

We introduced also the huge challenge farmers will face ensuring food security for the future generations, and how IoT comes handy for this mission, pushing farming to the next level: the *Smart Agriculture*.

# Chapter 2: Specification and design

After defining the main concepts of IoT and smart Agriculture in general, we will focus in this chapter with more details on a real application of smart agriculture. We present in the first place the requirements of our solution then its design in a second place.

## 1. Introducing the project

We are dealing with a farmer owning a vineyard of *Château Luchey Halde* at *Pessac, Bordeaux* in France *(a vineyard is a plot of land dedicated for grape cultivation)*. He wants to have a technological solution which will allow him to control his vineyard more efficiently in order to save time, energy, resources, and money and also improves the quality of grapes for a better production.

In order to have a more precise farming and a smarter agriculture, our client will need a device allowing him to have accurate measurements upon various environmental parameters and making adequate actions and optimal decisions based on it.

A way to satisfy the requirements of our client is in an IoT solution: sensors are needed for taking various measurements and sending them to a processing unit through a communication device. The processing unit will extract information from the collected data and take adequate decisions.

## 2. Vineyard monitoring

The grapes' quality is a very important element to our client willing to have the best possible production. This quality is in fact influenced by various elements among them we have two main axes: micro and macro climatic conditions of the environment.

Macro environmental elements are common elements to a particular location at a particular time. For example, the average temperature or rainfall in a farm at a given season, or the quantity of fertilizer that can be applied to the field are considered the macro climate for the vineyard.

In the other side, micro environmental elements are unique to each individual plant or to each small set of plants. Plant-to-plant spacing in a row is a very important micro environmental factor which will impact different phenotypes later during the development. Humidity and evaporation for each plant are also micro climatic elements.

Among the important ambient parameters needed for a healthy plant we have: light, temperature, relative humidity and wind.

### 2.1. Light

Grapevine requires high values of light for correct flowers development. It's due to the photosynthesis that leaves will transform light in order to produce sugar for the fruits. The more solar radiation we have, the more sugar accumulation will be in the grapes, which makes the fruit sweeter and more fragrant.

### 2.2. Temperature

Together with solar radiation, the temperature is an important climatic factor for the growth of grapevine. The high temperature will decrease the amount of organic acid inside the grapes. Light increases sugar accumulation and temperature decreases acid accumulation. Both are important factors for grapes maturity, thus for fruit quality.

In the other side, low temperatures are a risk for the plant. It damages the newly growing buds, which will cause a delay in leaves and flower development as well as in fruit maturity. Low temperature can also cause the death of the whole plant, and thus a severe damage to the field.

### 2.3. Relative humidity

Humidity is the amount of water vapour (moisture) in the atmosphere and it is expressed by gram of moisture per cubic meter of air. Relative humidity is the amount of water vapour in the atmosphere relative to a given temperature and expressed as a percentage of the amount of water from the total amount that could be held at the same temperature. Relative humidity is used for weather forecasting as it affects how temperature can be felt. [18]

When saturation level is reached for relative humidity, water vapour in the atmosphere condenses resulting in water drops over the leaves know as *dew (figure8)*. During the hot seasons (spring and summer periods) the dew is the last thing a farmer wants to see in his filed: heat and humidity will facilitate the development of some dangerous fungi infecting the grapevine. Some fungi will require a plant removal to prevent serious issues, others will spread from plant to another, causing severe damages to the whole filed.



*Figure 8: the Dew on a grapevine leaf*



*Figure 9: the Mildew fungi on a grapevine leaf*

[18] Ventilation for designing better buildings – September 2014

### 2.4. Wind

At the time of flowering, the wind favours the pollen spread. It helps to prevent the deposit of ice crystals, knows as the *hoarfrost*, which may occur overnight. Wind contributes also in ventilation which decreases partially the humidity around the plants.

Another aspect to take in consideration about wind is the evapotranspiration phenomenon. A simple way to define evapotranspiration is as being a combination of evaporation and transpiration. While evaporation is linked to water movement from the soil and earth to the atmosphere, transpiration is about water movement from the plant itself to the atmosphere. The following figure[19] illustrates this phenomenon:

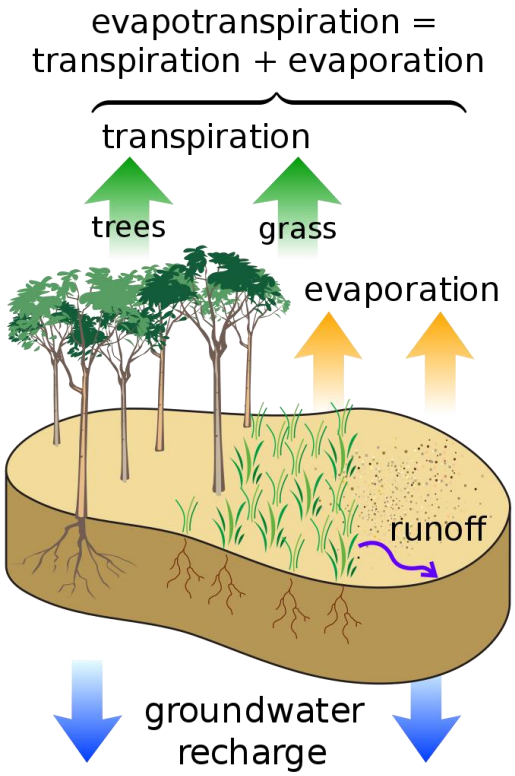



*Figure 10: the evapotranspiration*

Wind increases the evapotranspiration in the vineyard which affects directly the water balance in the field between the soil and the plant. To avoid drought problems or any other water-stress related issues, irrigation must be performed conveniently.

## 3. Project objectives

As we have seen in the previous section, controlling the environmental parameters impacts directly the grapes quality. In order to ensure the best possible quality, the proposed solution will allow our client to have control over the various parameter.

[19] By M. W. Toews - Own work, CC BY 4.0

Temperature and humidity are the priority of our client. We will provide him control over the soil and the air for temperature and relative humidity. During summer 2017, a group of students at ETS, in collaboration with University of Bordeaux, has developed a first prototype for our client. [20]

The prototype was built based on an Arduino board, with sensors for temperature and humidity monitoring, a dust sensor, a GPS, and a LoRa module for communication with a gateway. The prototype has been sent to France for test.

Our mission for summer 2018 is to build a second prototype since the first one is no more available at ÉTS, to enhance its existing functionalities, and to adapt it to the new needs of our client. He requires new functionalities able to prevent or detect critical threats to the grapevine health. Pests and fungi are considered among the main threats.

Our new prototype must include a function to identify or detect these threats. We propose to include a camera module in the system. Image processing techniques will help our client:

- Have control over the plant development to follow its growth.
- Identify diseases at a very primitive stage so that curing is still possible.
- Detect harmful insects (known as *pests)*.

As mentioned in their report, the previous team faced a technical problem:

*"The light sensor is not included in the final prototype due to limited memory resources in the microcontroller. If light sensor is added to the system, the memory will be overwhelmed to a 98% […] and in such conditions, the system is not stable and no LoRa communication is possible".* [21]

Being aware of such a technical limitation, and knowing that our second prototype is subject for future improvements, additional components and/or functionalities, our first goal is to revise the technical design and choose properly a new platform able to support the new needs of our client.

Another objective of this project is to make the prototype a modular system: it can provide a particular set of sensors with or without the camera, the camera alone, or the whole modules.

The first advantage of making a modular system is a financial one: different nodes will be spread all over the vineyard, and they do not all need to be exactly the same nodes. They can have different sensors depending on their location and other parameters. The second advantage of this improvement is energy consumption.

The final prototype we will make must be fully functional.

[20] « *Rapport de conception* ELE795 » by Philippe Chabot, Vincent Grosselin and Xavier Mercure-Gagnon
[21] « *Rapport de conception* ELE795 » by Philippe Chabot, Vincent Grosselin and Xavier Mercure-Gagnon

# 4. The sensor node design

A sensor node can be composed of one or more sensors and a communication module for data transmission. Basic sensor nodes are the simplest types: they don't store or manipulate the measurement sensed in any way, they simply send it to another node in the network. On the other hand, we have data nodes which are able to manipulate the data. Since we plan to make modular nodes, we are dealing with both types: simple and data nodes.

## 4.1. Choosing the platform

The previous prototype was built on an Arduino Nano platform, based on an ATmega328 microcontroller with only 2kB of SRAM.[22] The image processing we suggested to add as an enhancement for the previous prototype is impossible to be performed with such platform, knowing that it failed to support a light sensor due to limited resources.

An 8-bits black and white image having the dimensions of 50x50 pixels will approximately weight 2.4 kB.[23] The actual platform is not even able to store such image. We need to start by finding a suitable new platform for our enhanced prototype.

The following table resumes various platforms we considered: [24]

| Platform | RAM memory | Power consumption | Price |
|---|---|---|---|
| BeagleBone Black Rev C | 512 MB | 0.75 W to 1.4 W | 54.95 $ |
| Raspberry pi 2 | 1 GB | 1 W to 1.25 W | 35.00 $ |
| pcDuino 3 | 1 GB | 1.25 W to 1.75 W | 59.95 $ |
| Acadia | 1 GB | 1.4 W to 2.25 W | 119.95 $ |
| IGEP v2 [25] | 512 MB | 2.2 W to 2.8 W | 185 $ |
| Raspberry pi zero [26] | 512 MB | 0.4 W | 5 $ |

*Table 1: Embedded boards comparison*

The raspberry pi zero *(figure 11)* is the chosen platform for our sensor node. It has enough RAM memory for our application and an affordable price compared to other more complicated embedded boards. Its processor ARM Cortex-A53 is suitable for the needed image processing. The raspberry pi zero has also the least power

[22] Arduino web site
[23] North star imaging "Size for a digital image"
[24] Single board computer benchmark, by Shawnhymel
[25] IGEP v2 - datasheet
[26] Raspberry pi zero – power consumption comparison, by Jeff Geerling – Novembre 2015

consumption lever compared to the rest of the boards, which will allow our client to save energy and reduce his costs.

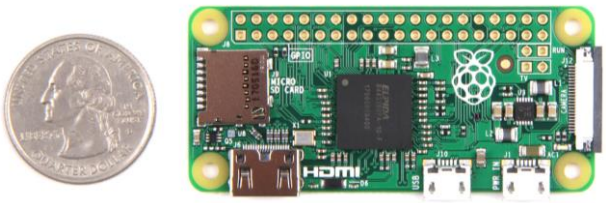


*Figure 11: the Raspberry pi board: a size comparison with a quarter dollar*

### 4.2. Choosing the operating system

Once we decide which platform to use, we need to choose a suitable operating system (OS) for our board.

There are many OSs that can be supported by this board. [27] [28]

#### *4.2.1. Raspbian Jessie*

The Raspbian Jessie is based on the famous and reliable Debian operating system used for critical equipment. It is a version of Linux that has been improved and better tuned over the years. It is the default operating system for the raspberry pi.

#### *4.2.2. RetroPie*

RetroPie is an operating system developed for gamers in order to enjoy classic video games on their tiny computer: the raspberry pi board.

#### *4.2.3. Kali Linux*

Kali Linux is the classic operating system of Linux used for security purposes. It comes with a built-in security toolset and is used classically for network penetration tests and hacking.

#### *4.2.4. Open Media Vault*

Open Media Vault allows the user to set up his raspberry pi card as a NAS device (Network Attached Storage). In a simpler way, it makes the user set up his own personal cloud.

#### *4.2.5. IchigoJam BASIC RPI*

IchigoJam BASIC RPI was inspired from the kids PC *IchigoJam* original edition. The OS is designed with the idea of making programming a funny and joyful experience. It makes it easy enjoying coding with basic language.

[27] Raspberry pi zero OS list
[28] Raspberry pi web site / Download page

#### 4.2.6. *Windows 10 IoT Core*

Windows 10 IoT Core is a modified version of the original desktop computers OS windows 10 optimized for smaller devices with or without an external display. Microsoft offers two version: Windows 10 IoT Core and Windows 10 IoT Enterprise.

#### 4.2.7. *Choosing the operating system*

The operating system we will choose for our project is the default Rasbian OS. It is a reliable OS that has proved its performance over the years to be chosen as the default OS for the whole raspberry pi family. It has also the advantage of having an available full documentation online and a large community compared to the other operating systems listed above.

Rasbian does not have an easy evident user interface like a windows 10 OS, so future improvements of the project will require people having good knowledge of Linux commands and terminal. Besides, Rasbian does not have real-time capacities, but since our project does not require any real time constraint, this will not be considered as a weakness in our choice.

### 4.3. Choosing the video/image capturing technique

Two main techniques can be used here:

- Use the CSI connector of the raspberry pi *(Camera Serial Interface) (Figure12)*. It has the advantage of being extremely simple for usage since many capture and shooting tools are already built-in functions on the card.
- Use a CCTV *(closed-circuit television)* camera connected with a coaxial cable. The camera will look out over the plants and will store photos and videos in a storage node.




*Figure 12: CSI connector on the raspberry pi (on the left)*
*and connecting a camera module on this interface (on the right)*

We decided to choose the first option, as it's the most intuitive approach and because various libraries are available to work with.

Among the objectives of our project - as defined by the client - is to detect the main threat of the grapevine (fungi and pests). Our prototype needs to take photos and analyze them in order to detect if any unhealthy signs are present. Image analysis can be performed in 3 ways:

- An employee from the vineyard will manually check the photos and analyze them looking for any particular signs of threat.
- Include an intelligent aspect such as a machine learning algorithm or a neural network concept in order to analyze the important amount of photos.
- Make image processing at the sensor node and only forward the final processed image to a gateway.

The first approach is time-consuming and not efficient at all. Our client owns a large vineyard. An important amount of photos will be taken overall the field and check every single one of them manually is time-consuming and an extra charge for our client to employ an additional worker.

The second approach is efficient in terms of result and time consumption. But in terms of complexity, it's a very complicated technique and requires important processing abilities and high-performance nodes, which means more expensive hardware for our client. Besides, it is a whole complete project on itself and not a simple improvement for the previous prototype.

The third and final approach is the most reasonable one. In terms of complexity, it is simpler than the second approach, and in terms of time and resources consumption, it is also better than the previous two approaches. And it does not add any significant extra charge for our client.

## 5. The Gateway design

The Gateway is the second node to be developed during this project. As we found out in the previous broad comparison *(made in section 4. The sensor node design)* the raspberry pi are the cheapest cards.

The Gateway node is the main node which will receive multiple data collected from all the sensor nodes spread on the vineyard. The Gateway requires higher processing capabilities and also a higher memory to deal at the same time with multiple data reception.

The following table[29] resumes the available raspberry pi models with the highest memory:

| Model | Memory | CPU Clock | Release date | price |
|---|---|---|---|---|
| Raspberry pi 2 | 1GB | 900 MHz | 01-02-2015 | 35.00 $ |
| Raspberry pi 3 | 1GB DDR2 | 1.2 GHz | 29-02-2016 | 35.00 $ |
| Raspberry pi 3 B+ | 1GB | 1.4 GHz | 14-03-2018 | 35.00 $ |

*Table 2: Raspberry pi models comparison*

[29] *« Comparatif des modèles de Raspberry pi »* by Social compare, August 2018

We choose a raspberry pi 3 *(figure 13)* as a platform for the gateway. The Model B+ was not available to use. For the OS, we will run the default Raspbian Jessie as what we have chosen for the sensor node.



*Figure 13: The raspberry pi 3*

## 6. The communication protocol

The sensor node we will build needs to communicate with the gateway in order to send to it all the data and measurements he managed to collect. Different approaches are available on the market and we need to choose the best suitable one for our prototype.

We can go for a wired or a wireless communication. A wired approach is not possible in our case: having cables spread all over the vineyard will definitely make any soil management, soil tillage, pesticide treatments, harvesting, or any other cultivation techniques that require a truck intervention, an impossible operation, or at least, a more time consuming and a less efficient operation. Trying to hide the cables underground or lift them above the soil at a secure distance can be a better alternative than spreading cables on the ground, but this will only be a lot of extra charges and additional not-needed costs for our client.

Since a wired approach is not possible, the wireless communication is always the best alternative when wired systems cannot be deployed on the field. As networks can be mainly divided into three levels, we need to choose a suitable technology among the following list:

- Personal Area Networks such as Bluetooth.
- Local Area Networks such as Wi-Fi.
- Wide Area Networks such as cellular networks.

The following table[30] [31] represents a comparison of various standards:

[30] "802.15.4 wireless for IoT developers" by Mike Vidales – Helium Blog, May 2017
[31] "11 IoT protocols you need to know" April 2015

| Technology | Frequency | Data rate | Range | Power usage | cost |
|---|---|---|---|---|---|
| Z-Wave | Sub GHz (900 MHz) | 40 kbps | ~ 30m | Low | Medium |
| Bluetooth/BLE | 2.4GHz | 1,2,3 Mbps | 50 to 150m | Low | Low |
| ZigBee | 2.4GHz | 250 Kbps | 10 to 100m | Low | Medium |
| WirelessHART | 2.4GHz | 250 Kbps | ~ 90m | Medium | Medium |
| Wi-Fi | Sub GHz, 2.4GHz, 5GHz | 150 to 200 Mbps | ~ 50m | Medium | Low |
| Weightless | Various | 1 Kbps to 10 Mbps | Up to 10 Km | Low | Low |
| LoRaWAN | Various | 0.3 to 50 Kbps | * 2 to 5 Km (urban)<br>* 15 Km (suburban) | Low | Medium |
| SigFox | Sub GHz (900 MHz) | < 1 Kbps | * 30 to 50 Km (rural)<br>* 3 to 10 Km (urban) | Low | Medium |
| 2G/3G | Cellular bands | 10 Mbps | * 35 Km max for GSM<br>* 200 Km max for HSPA | High | High |
| LTE Cat 0/1 | Cellular bands | 1 to 10 Mbps | Several Km | Medium | High |
| NB-IoT (narrowband IoT) | Cellular bands | 0.1 to 1 Mbps | Several Km | Medium | High |

*Table 3: Existing wireless solutions overview*

We have already a vast choice of wireless technologies. We need to choose an adequate solution that will ensure us reliable communication at a lower cost.

The first five options Z-Wave, Bluetooth, ZigBee, WirelessHART and Wi-Fi have a limited range not enough to cover the whole large field of our client. Personal Area Networks (PAN) and Local Area Networks (LAN) are restricted technologies and do not satisfy our goal. We need a Wide Area Network (WAN) solution.

Cellular networks provide a good range and coverage thanks to the existing infrastructure. They are also capable of transmitting an important amount of data especially the 4G. However, maintaining a cellular network has a very high cost and they have a high power consumption which is not acceptable to our client. The 2G/3G, LTE, and NB-IoT technologies will not be used for our prototype.

The remaining options are Weightless, SigFox and LoRaWAN. Among all of them, one technology is getting a lot of interest in the IoT field recently: the LoRa. It allows low power consumption for nodes' battery economy and a long-range communication.

Weightless has a lower coverage compared to LoRaWAN. SigFox has similar characteristics to the LoRaWAN, with even a better coverage. However, what makes LoRa a better option is its open protocol. A SigFox network will always be connected to their infrastructure, and being dependent on them can raise some security and privacy issues for our client data transmission. In the case of LoRaWAN, we can set our own private infrastructure for our own application. [32]

LoRaWAN is a very promising technology. While there are other solutions that provide similar characteristics, LoRa seems to be the best option for us due to its open nature and the important community supporting it. For all the reasons mentioned, LoRaWAN is our choice for the communication protocol.

LoRaWAN can be called in a simpler way as just LoRa. While technically speaking there is a difference between the two words, we will refer in the rest of this report to both of them as just LoRa. LoRaWAN is a wireless wide area network, LoRa is the modulation technique used in the physical layer of the LoRaWAN network.




*Figure 14: Lora technology logo*

## 7. The flow chart

The design we choose for the system is to make it modular, which gives flexibility to the client deploying various nodes with various sensors. The following figure represents the flow chart of the system:

[32] "Is Lora the game-changer for IoT?" April 2016

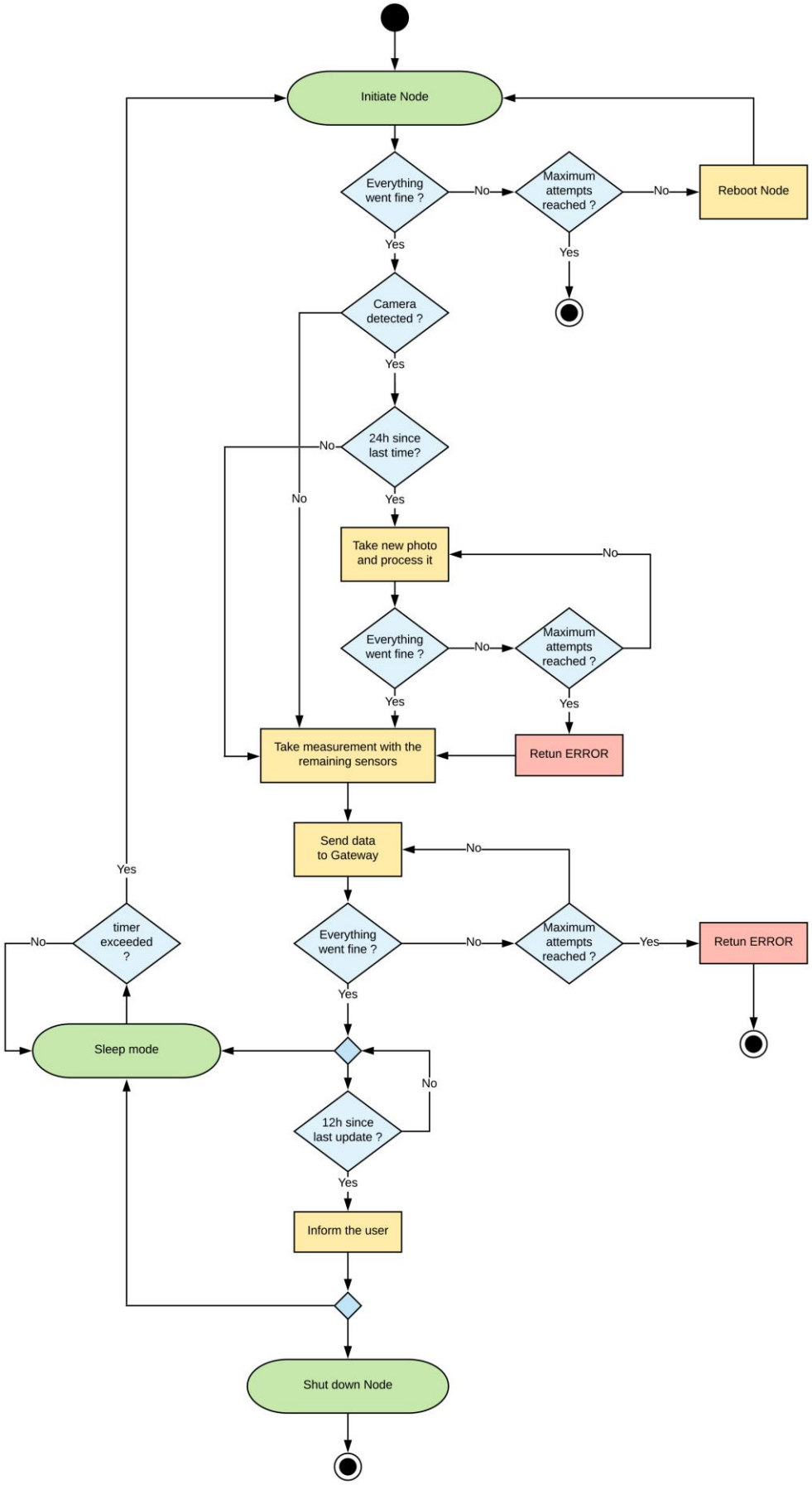



*Figure 15: Flow chart of the system*

The first thing to do when the system boots and in case of no boot error, is to check for the camera existence. A photo will be taken once a day. After taking all the measurements and in case of no error in data transmission to the gateway, the system checks if 12 hours have been exceeded since the last update so that the user is informed two times a day.

The sensor node switches to sleep mode after each cycle to restart again the whole process when the timer runs out. The user when updated with new data, he can switch off manually the sensor node if he no more needs it, otherwise, it will switch to sleep mode.

The system has 3 exit points:

- When it fails to start to boot properly, and after a certain reboot attempts if the problem is still not solved, the system will exit with a fatal boot error and the node will be shut down.
- When the LoRa transmission fails and the node can no more send properly any error message to the gateway, if the problem is still there after a certain reboot attempts, the node will be shut down.
- When the user manually shuts down the node.

For the first 2 cases, we are dealing with a critical system errors and it requires a human intervention to fix the problem. The gateway will notice the long absence of the sensor node from the network and it will send an alert message to the user informing him about a fatal system error.

For the third case, it is the user's choice to shut down the node, the gateway will be informed and will ignore the absence of this particular node until reactivated again.

## Conclusion

In this chapter, we presented the requirements of our client and what he expected to have as a result. A previous prototype has been developed last year and faced a technical issue in the number of sensors they could set due to limited resources of the Arduino platform they used.

Our mission was to make a full technical revision for the previous design in order to support the future enhancements and to build a new prototype fully functional matching the new requirements of our client.

We presented in this chapter different available options in the market, and we proved why the choice we made for the new design has better potential for future enhancements.

# Chapter 3: Implementation and results

After defining the project's goals and the necessary elements, from the problematic and needs up to design and technical choices, we are left with the final step: implementing a prototype.

This chapter presents the working environment and the output we got from every step.

## 1. Working environment

The tiny computer *Raspberry pi* has been developed with the idea of encouraging more people to get into coding and computer science. It is meant to be as affordable as possible and make young people have fun building their own projects. To develop our project on this board, many programming languages can be used :

- C/C++ :

C was created in the early 70s by Dennis Ritchie and became later the standard programming language in UNIX systems. C++ is an enhancement to the C. While C is a procedural programming language, C++ is an object-oriented language and intended to be more general. C and C++ are a good start for embedded systems.

- Python :

Python is one of the primary languages hosted by default in all the raspberry pi boards. The "pi" itself on the "raspberry pi" was named after Python. While Python was not initially used for embedded systems, developers start to use it more seriously for such applications. More and more libraries are made available to make Python as useful for embedded systems as C or C++.

- ERLANG :

It is used for severe critical missions where no failure can be tolerated. With ERLANG, the program can be run in several machines so that if one computer fails, another one will back up in a way that the system never goes down.

- Scratch :

Scratch is a very simple language used for entry-level. It comes also as a standard with the Raspbian OS distribution.

- Etc…

Other programming languages can be used for the raspberry pi, such as Java, JavaScript, Perl, HTML5, etc…

We don’t need ERLANG since our application does not require any extreme availability constraint. Among all the others, C/C++ and python are the most popular languages with an important community and documentation available everywhere.

C++ is the chosen language. It will be mainly used for the sensors. We will use also Python for shooting photos with the camera and a Shell script will be the last step to arrange the whole code together.

## 2. Setting up the sensor node

The raspberry pi board is able to communicate with an external device or circuit through its GPIO *(figure 15)* abbreviation of *General Purpose Input/ Output.* Depending on its configuration, a GPIO pin can be an input (receive a signal) or an output (send a signal).

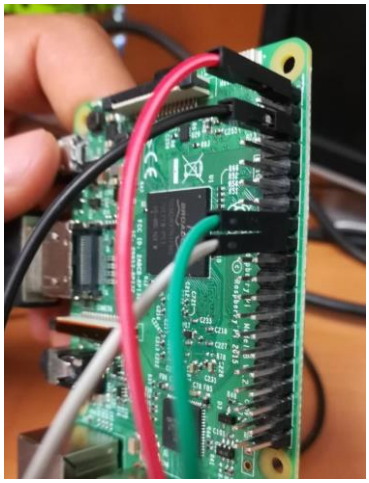


*Figure 16: four wires (red, black, green and white) connected to GPIO pins*

GPIO pins are managed by the OS Core of the electronic board. Apart from the standard OS core library for GPIO, other external libraries at a higher level are available to make GPIO programming simpler and easier. In addition to simplifying pin management, these libraries make electronic devices from different manufacturers and different architectures able to communicate together.

In order to be able to add various sensors into our raspberry node and make them communicate with the board, we need to start by downloading a GPIO library to prepare the work environment. The GPIO library available for raspberry boards is the WiringPi library.

The WiringPi is written in C by Gordon Henderson and is supported by all the raspberry pi versions. This library comes with an additional program for easier onboard GPIO management. It is called *gpio* and can be called inside the scripts to manipulate the GPIO pins and set inputs and outputs pin.

The set of functions coming with the WiringPi library allow the raspberry pi GPIO pins to be accessed independently of which programming language used. It has been written initially for the C, it is also available for Python and Ruby.

WiringPi uses the following numbering for the GPIO pins[33]:

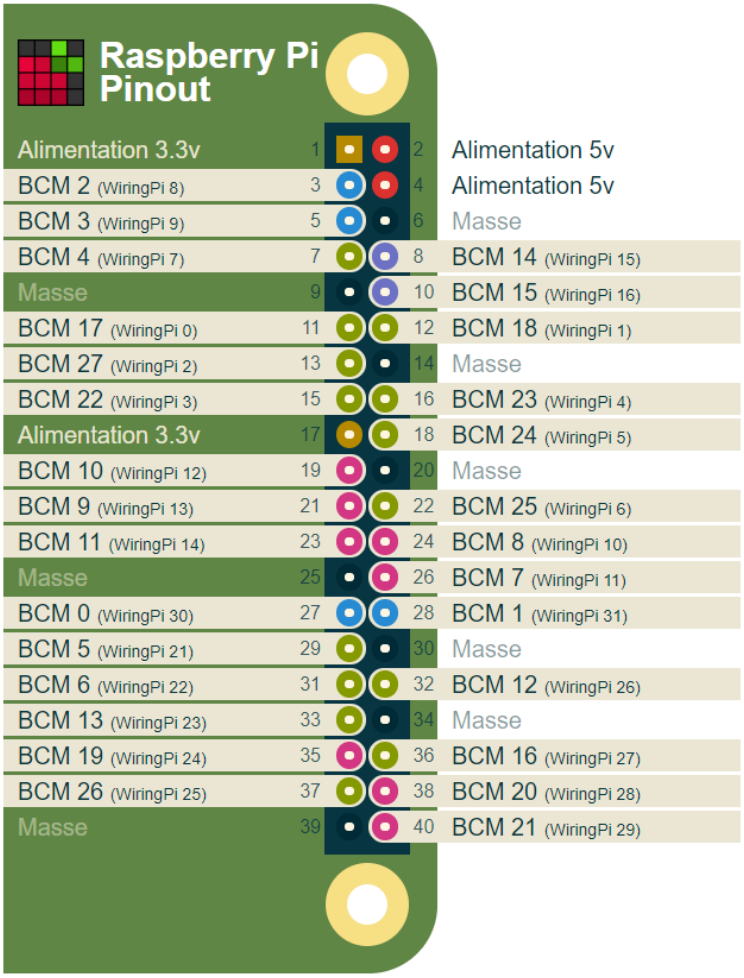



*Figure 17: WiringPi pins numbering*

The numbering of the GPIO pins according to the WiringPi library as shown in figure 16 is what we will use later to assign every sensor to its correspondent pins.

## 2.1. The DS18b20 sensor

The DS18b20 sensor *(figure 17)* is a digital thermometer providing temperature measurements in Celsius. We use it for soil temperature monitoring.

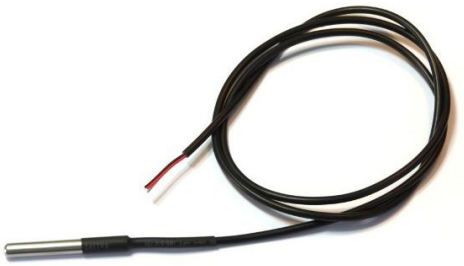


*Figure 18: DS18b20: a temperature sensor for the soil*

[33] Illustration taken from https://fr.pinout.xyz/pinout/wiringpi

It has 3 wires to be connected to the raspberry node: the black wire is to be connected with a ground GPIO pin, the red wire with a 3.5 V alimentation GPIO pin, and the yellow wire for data will be connected to a general purpose pin. We choose to connect it to the pin number 15 and program it as an input pin so that the raspberry pi can receive a signal from the sensor through this pin.

The following photo shows our DS18b20 sensor connected to the raspberry pi.



*Figure 19: a DS18b20 sensor connected to the raspberry pi zero*

Running the DS18b20 correspondent C++ code on the raspberry pi zero will give us the following output:

```
pi@raspberrypi:~/single-sensor-tests/DS18b20 $ sudo ./temp_test
0. Temperature is around 23.1875
1. Temperature is around nan
2. Temperature is around 23.1875
3. Temperature is around 23.1875
4. Temperature is around 23.1875
5. Temperature is around 23.1875
6. Temperature is around 23.1875
7. Temperature is around 23.1875
8. Temperature is around 23.1875
9. Temperature is around 23.1875
```

*Figure 20: DS18b20 sensor output sample*

## 2.2. The SKU: SEN0193 sensor

The SKU: SEN0193 sensor (figure 20) is a soil moisture sensor that measures the moisture level by capacitive sensing instead of resistive sensing like most of the other sensors available on the market. It is built out of corrosion resistant elements which give it an excellent lifespan.




*Figure 21: Soil moisture sensor*

It has 3 wires to be connected to the raspberry node: the black wire is to be connected with a ground GPIO pin, the red wire with a 3.5 V alimentation GPIO pin, and the blue wire for data will be connected to a converter: the soil moisture sensor provides an analogue output, however, the raspberry pi can only read digital inputs. An analogue to digital converter (ADC) is needed for this sensor. The blue wire for data will be connected to an input channel of an ADC. We will use the MCP3008 converter.



*Figure 22: Analogue to digital converter (the MCP3008)*

The following photo shows our soil moisture sensor connected to the raspberry pi zero through an ADC connected to a breadboard.

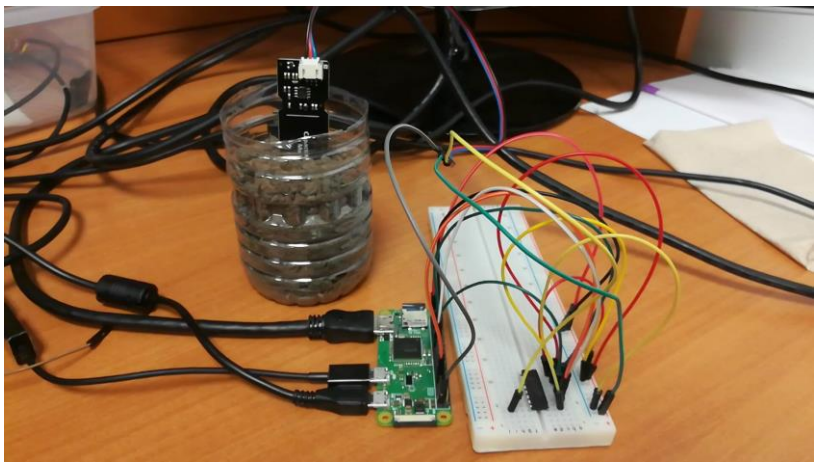


*Figure 23: Soil moisture sensor connected through an ADC to the raspberry pi*

The soil moisture sensor requires a calibration in order to provide us with correct measurements. We need to define where are the limits of extreme *"Dry"* value and the extreme *"Wet"* value.

We record the voltage detected by the probe when it is exposed at the air. This will be the boundary value of dry soil (relative humidity equal to 0%). The sensor detects every time an individual value slightly different from the previous one. It is due to its sensitivity detection. We made an average value over 100 outputs and we record the dry boundary as $B_{dry}$ = 4.317864 V.

Next, we repeat the same experience but this time by inserting the probe in a cup of water. The value recorded will be the extreme boundary value of a wet soil (relative humidity equal to 100%). We compute the average of 100 distinct outputs and we record the wet boundary as $B_{wet}$ = 1.93877 V.

Once we have the 2 extreme boundary, the between values will be divided in different ranges. Let's call R the range between the 2 boundaries:

$R = B_{dry} - B_{wet} = 4.317864 – 1.93877 = 2.379094$ V

We choose to define 3 regions as follow:

- Very wet soil = $[B_{wet} , B_{wet} + \frac{1}{3} R]$
- Wet soil = $[B_{wet} + \frac{1}{3} R , B_{dry} - \frac{1}{3} R]$
- Dry soil = $[B_{dry} - \frac{1}{3} R , B_{dry}]$




*Figure 24: Different measurement regions for the soil humidity sensor calibration*

The code we developed does not include the explicit values of the inner boundaries. We rather kept the R parameter for region definition. This will make any future optimization easier. For example, If we will need 5 regions in the future (let's say very dry, dry, normal, wet and very wet) we simply change the 1/3 to a 1/5.

Monitoring the soil moisture allows our client to control the water content of the soil so that irrigation will be optimized, which leads to saving water and reducing resources consumption for our client.

An output sample of the soil moisture sensor is shown in the following screenshot:

```
pi@raspberrypi:~/single-sensor-tests/soil_rh $ sudo ./soil_rh_test
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.99512V
  Dry
Soil RH = 4.49707V
  Dry
Soil RH = 3.98438V
  Dry
Soil RH = 3.98926V
  Dry
Soil RH = 4.49219V
  Dry
```

*Figure 25: SKU:SEN0193 sensor output sample*

### 2.3. The DHT22 sensor

After having control over the temperature and humidity of the soil, we need also to allow our client tracking the air conditions surrounding the grapevines.

DHT22 is a basic low-cost sensor for humidity and temperature of the atmosphere. It provides a digital output, so no ADC is needed for conversion like the soil moisture sensor.




*Figure 26: a DHT22 sensor*

Fungi develop in a humid environment, tracking the humidity level in the surrounding atmosphere will allow our client to be aware if an infection risk is present due to humidity values exceeding a threshold.

The DHT22 sensor has 3 connectors: the $V_{cc}$ pin will be connected to the 3.5V alimentation pin of the raspberry, the GND will be connected to a ground pin, and the data pin will be connected to a general purpose pin. We chose pin 8 and program it as an input pin.

The following figures show our DHT22 sensor connected to the raspberry pi and a sample of the output.

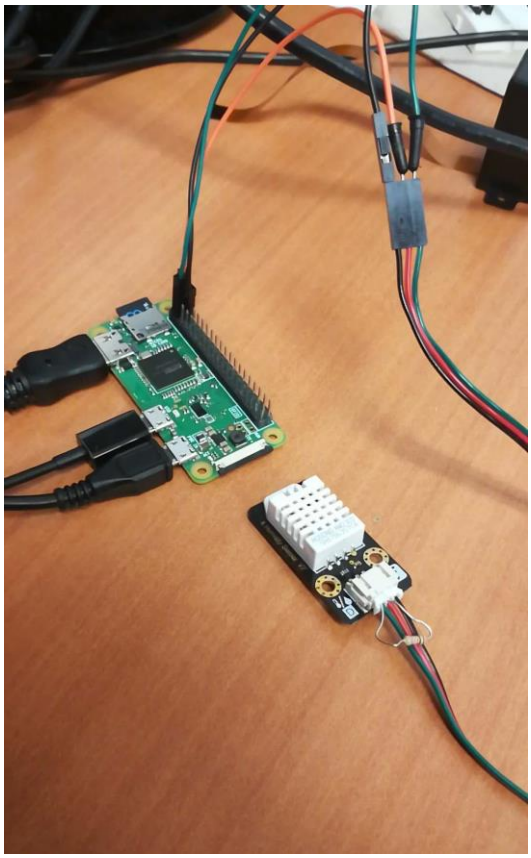


*Figure 27: the DHT22 sensor connected to the raspberry pi*

```
pi@raspberrypi:~/single-sensor-tests/dht22 $ sudo ./dht22
1e900e6d0
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
Temperature is 23°C, humidity is 48.9%
```

*Figure 28: DHT22 sensor output sample*

## 2.4. The raspberry pi Camera module

We used the raspberry pi camera module v2. It has a Sony sensor the IMX219 8-Mpx. It is able to take HD videos as well as good quality photos. It works with all the raspberry versions and can be accessed through many libraries developed by the raspberry community or third parties.

A protective shield is used for our camera module as shown below. The camera is connected to the raspberry pi through the SCI connector.

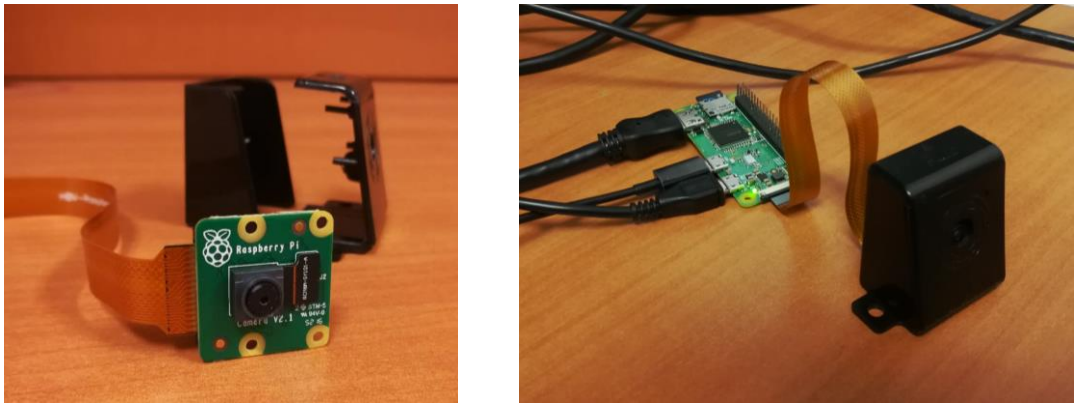



*Figure 29: A camera module v2.1 with a protective plastic shield is connected to the sensor node*

We used python library *Picamera* to access our camera. In order to shoot appropriate photos, the code we developed will ensure the camera keeps waiting 3 seconds before shooting, so that light sensor is adjusted properly and the final image will be in good quality with realistic colours.

The main function of the camera we included is to track any signs of a health threat to the plants. Black rot and mildew are the main fungi threats we need to detect.




*Figure 30: Mildew (to the left) and Black rot (to the right)*

Mildew and Black rot are brown and grey circles over the leaf. The code we used will start by shooting a photo. Once we have an input image, it will be transformed from the RGB (Red Green Blue) to the HSV (Hue Saturation Value) space. Using HSV space makes an object with a certain colour able to be detected while reducing the noise caused by outside light. [34]

[34] Image classification with HSV Color Model Processing, by Jeefri A. Moka – October 2017

The second step will be to reduce the green level so that fungi signs can be better distinguished from the leaves. A black and white filter enhancing fungi signs to be brighter compared to the rest of the image will be then applied.

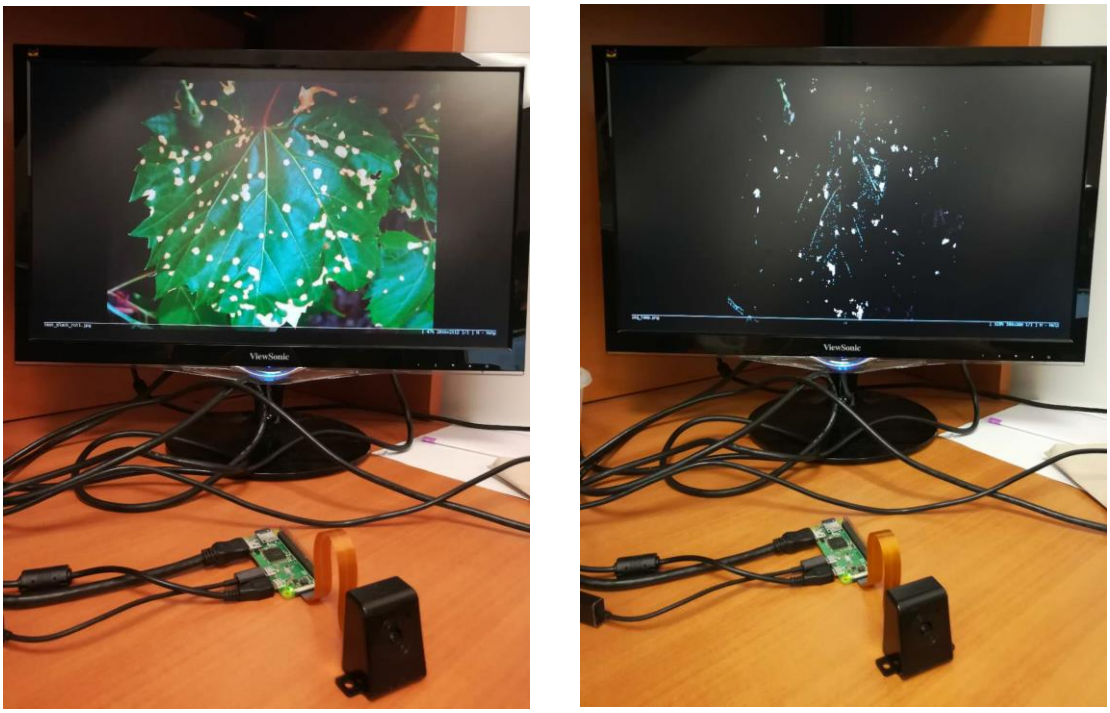


*Figure 31: an input image (to the left) VS the processed image (to the right)*

The bandwidth of a LoRa communication is limited. The final photo will be compressed so that it can be sent easier without any risk of network congestion in case of multiple sensor nodes sending simultaneously their images to the gateway.

# 3. Setting up the Gateway

## 3.1. The LoRa module

Our gateway is built on a raspberry pi 3 board. We have different options available for which LoRa module to use.

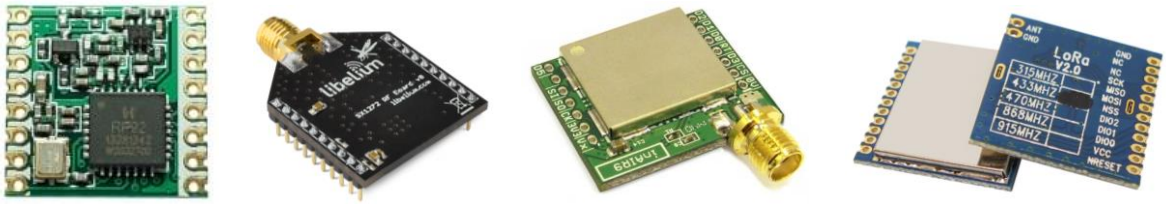



*Figure 32: Different LoRa radio modules supported by raspberry pi*

The previous figure shows in order from left to right:

- HopeRF RFM92W/95W
- Libelium LoRa
- Modtronix inAir9/9B
- NiceRF LoRa1276

The RFM95W, inAir9/9B and NiceRF LoRa1276 use the Semtech SX1276 transceiver chip, which allows ultra-long range spread spectrum, high resistance to interference and reduced energy consumption.

We used the Modtronix inAir9 module. It has the advantage of having header pins already soldered and with an affordable price. We connect this radio module to the raspberry pi as described in its datasheet. The following photo shows our gateway connected to the Modtronix inAir9:

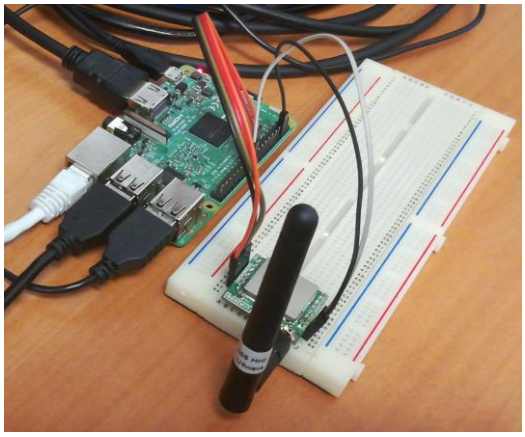


*Figure 33: the LoRa module connected to the raspberry pi 3*

The OS we run on the gateway and its main configuration were as described by Professor Congduc Pham in his documentation. [35]

### 3.2. LoRa transmission test

We connect a second LoRa module (Modtronix inAir9) to our sensor node. We will use the DS18b20 sensor for data transmission test.

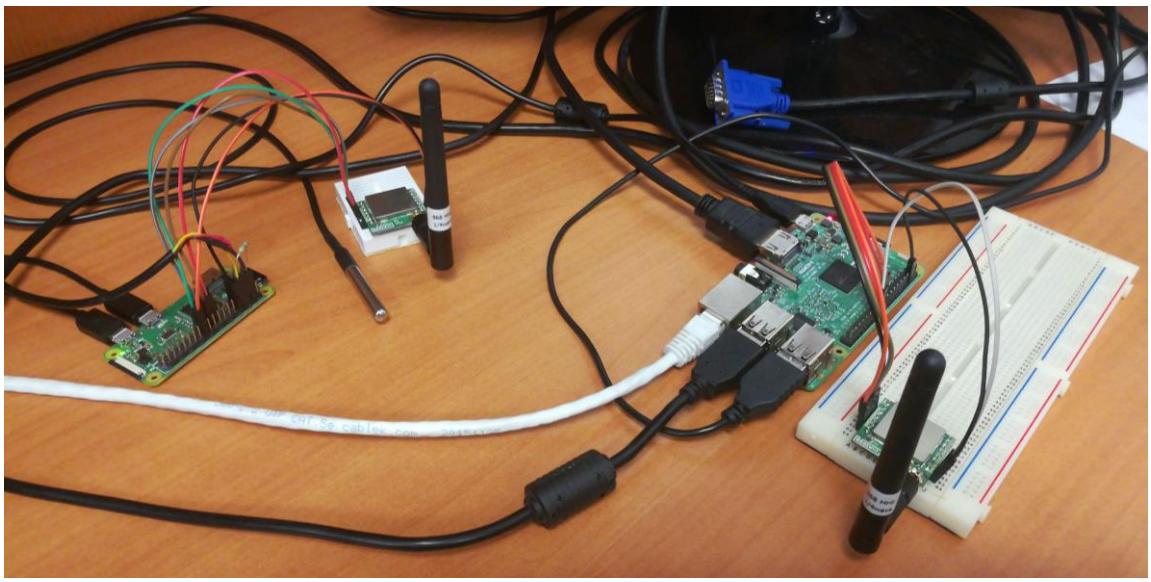


*Figure 34: the Sensor node and Gateway configuration (1)*

[35] Low-cost LoRa IoT framework, by C. Pham

For configuration, the Gateway will be connected directly to an external screen while the sensor node will be accessed remotely by an SSH session.

We used the MobaXterm software for remote access. MobaXterm provides all the important remote network tools (SSH, VNC, FTP, X11…) and Unix commands (grep, cat, ls, bash, awk …) to Windows. It is a good toolbox for remote computing.

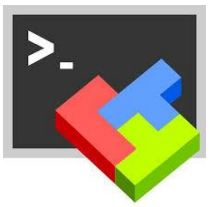


*Figure 35: MobaXterm software logo*

The following photo shows the configuration we used for our test:

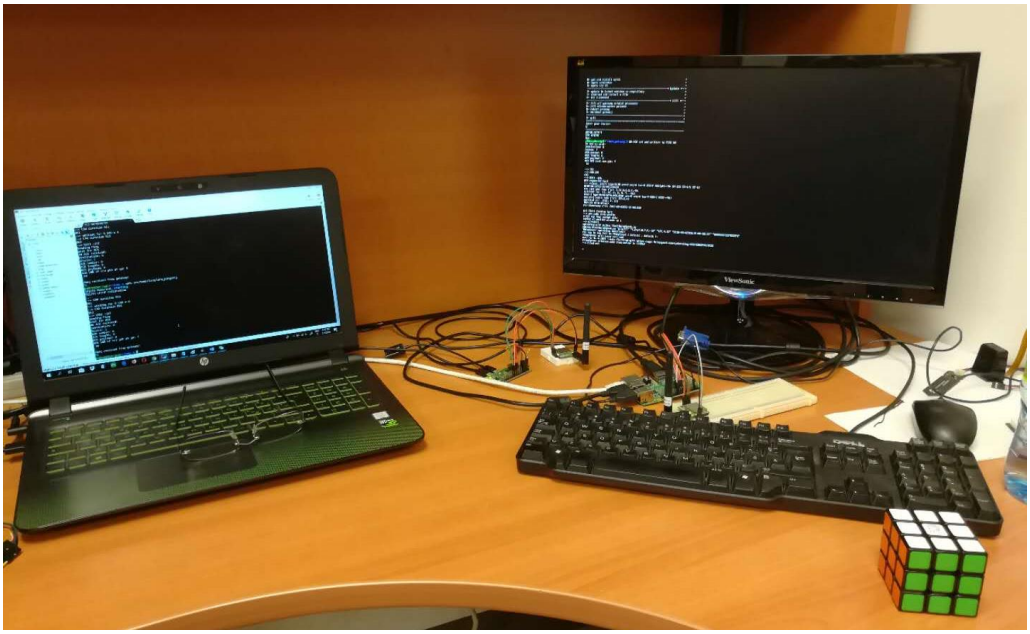


*Figure 36: the Sensor node and Gateway configuration (2)*

To the right, we have the gateway accessed directly with an external display. To the left, we have the sensor node accessed remotely from my PC with an SSH session using the MobaXterm software.

In the first step, we compile a lora_gateway program to start our gateway in standalone mode. The default settings are kept as described in the documentation.[36] We use maximum power transmission and LoRa Libelium mode 4 having the following parameters:

- Frequency: 865.2 MHz
- Bandwidth BW = 500 KHz
- Spread factor SF = 12
- Coding rate CR = 4/5

[36] A DIY low-cost LoRa gateway, by C. Pham

```
> sudo ./lora_gateway
Power ON: state 0
LoRa mode: 4
Setting mode: state 0
Channel CH_10_868: state 0
Power M: state 0
Get Preamble Length: state 0
Preamble Length: 8
LoRa addr 1 : state 0
SX1272/76 configured as LR-BS. Waiting RF input for transparent RF-serial bridge
```

*Figure 37: Gateway running in standalone mode*

After successfully running the Gateway in standalone mode, the next step is to check if the communication between the Gateway and the sensor Node is possible or not. We run a test code called “LoRa Ping-Pong”.

The sensor node sends a ping to the Gateway and waits for an answer. If everything works correctly, the Gateway will receive the request and reply by a pong to the sensor node. If the communication will fail, after a timer run out and still no answer received, the sensor node will display an error message “no pong received from the gateway”.

```
pi@raspberrypi:~/temp $ sudo src/temp/lora/lora_pingpong
SX1276 detected, starting.
SX1276 LF/HF calibration
...
--> CAD duration 593
OK1
--> waiting for 8 CAD = 0
--> CAD duration 594
OK2
--> RSSI -127
Sending Ping
wait for ACK
## ACK received:
Destination: 8
Source: 1
ACK number: 0
ACK length: 2
ACK payload: 0
ACK SNR of rcv pkt at gw: 7
 ##

Pong received from gateway!
^C
pi@raspberrypi:~/temp $
```

*Figure 38: A ping-pong test between the sensor node and the gateway (sensor node perspective)*

The previous screenshot is an output sample from the sensor node accessed remotely by an SSH session. We highlighted the most important information in red rectangles. The meaning of the whole output is as follow:

- The code starts with a calibration for the SX1276 library in order to make a LoRa transmission.
- CAD is the Channel Activity Detection. CAD duration depends on the spread factor used as explained in the datasheet. [37]
- The RSSI is the Received Signal Strength Indication.
- Ping is sent, and the sensor node is waiting for the acknowledgement ACK from the gateway.
- Information about the source and destination with ACK packet format are displayed.
- Pong received from Gateway! That was the confirmation our communication has been successful.

From the other side, the Gateway is also receiving a signal from the sensor node and it is displaying information. The following photo is what has been displayed on the external screen connected to the Gateway:

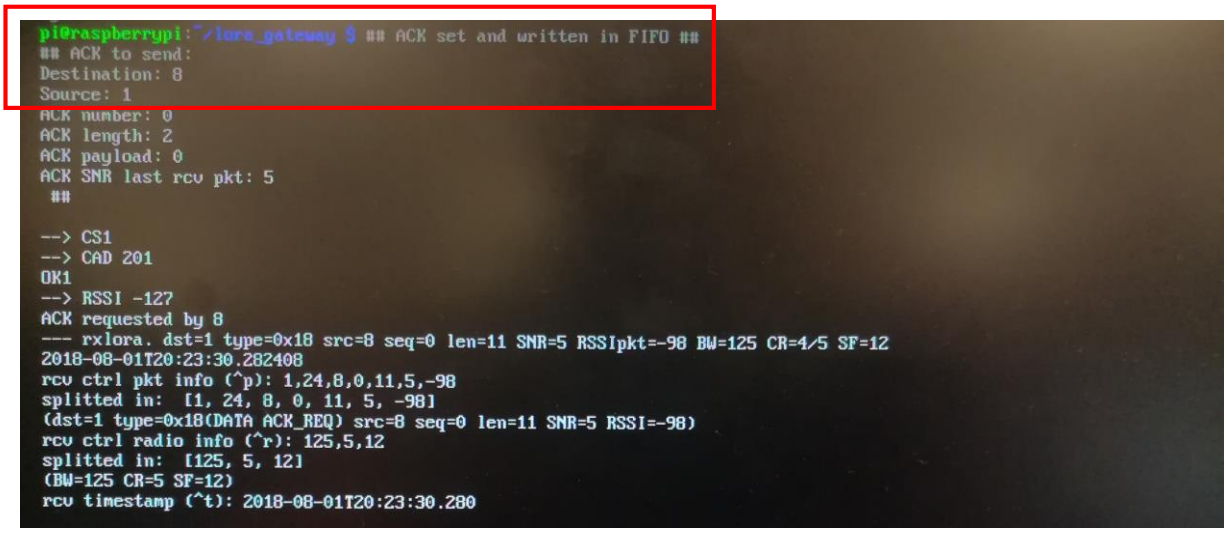



*Figure 39: A ping-pong test between the sensor node and the gateway (gateway perspective)*

We highlighted the most important information in red rectangles. The meaning of the whole gateway output is as follow:

- ACK has been written and going to be sent with the displayed information of the destination, source, and ACK packet format. Other parameters like the CS, CAD and RSSI are displayed.
- The received control packet information (^p): 1,24,8,0,11,5,-98 is a compact format of the previous information line : destination ID = 1, type =0x18 converted to decimal value 24, source ID = 8, sequence =0, length of the sequence = 11, SNR =5 and the RSSI level = -98.
  Same information is written in the next line as [1, 24, 8, 0, 11, 5, -98].
- The received control radio information (^r): 125,5,12 is the information about bandwidth used BW=125, coding rate CR = 4/5 and spread factor SF=12.
  The same information is written in the next line as [125, 5, 12].
- The timestamp is written in the ISO format (^t): 2018-08-01T20:23:30.280 which is read as August 1st, 2018 send at 20:23 *(and 30.280 seconds)*.

[37] Reading channel RSSI during a CAD, by SEMTECH

Once the communication was established correctly, we will test a data transmission from the sensor node to the gateway.

We will use the soil temperature sensor as a test. The measured temperature by the sensor node will be sent to the gateway through the LoRa module.

We run the temperature sending code from the sensor node. Below is a sample of the displayed information on screen:

```
pi@raspberrypi:~/temp $ sudo ./temp
Temp started
SX1276 detected, starting.
SX1276 LF/HF calibration
...
Client ID is b8:27:eb:1f:45:4d
--> CAD duration 604
OK1
--> waiting for 8 CAD = 0
--> CAD duration 604
OK2
--> RSSI -127
next is bogus:
Scratch = 0xffffffffffffffff
Temperature is around -0.0625
Scratch = 0x7f100cff7f464b017c
Temperature is around 23.75
Sending temperature (ping)
wait for ACK
## ACK received:
Destination: 8
Source: 1
ACK number: 0
ACK length: 2
ACK payload: 0
ACK SNR of rcv pkt at gw: 8
 ##

Pong received from gateway!
^C
pi@raspberrypi:~/temp $
```

*Figure 40: Temperature transmission (sensor node perspective)*

We have a similar output to the previous code, with a new information about the temperature sensed (23.75 °C) being sent while waiting for ACK. In the end, we have a received pong from the gateway which means our transmission was successful.

The client ID we highlighted in a blue rectangle is the MAC address of the sensor node. Since the MAC address is a unique identifier, this will allow the gateway to know exactly which node is sending which data, and allow our client to know which area of his vineyard is concerned in case of a problem alert.

Even though during our project we are working on a single sensor node for this prototype, the future application will include a multiple sensor nodes spread all over the field, and a distinction between each node will be required to avoid any mess.

The client ID displayed in the previous output sample is b8:27:eb:1f:45:4d and can be easily compared to the MAC address of the sensor node by using the *ifconfig* command:

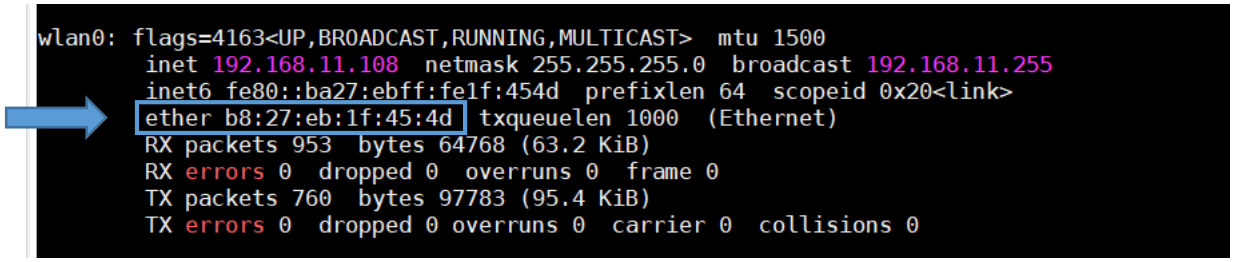



*Figure 41: MAC address of the sensor node used as the transmission ID*

At the same time from the other side, the gateway is also receiving data from the sensor node:

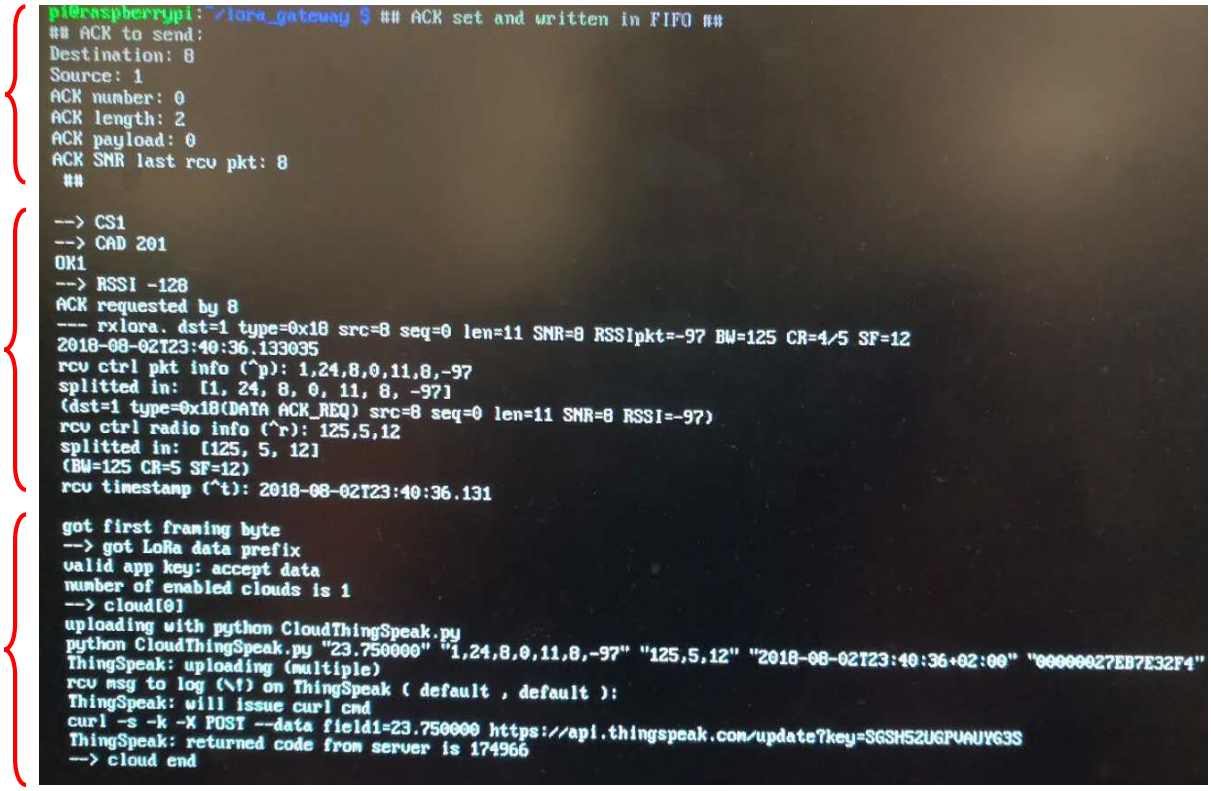



*Figure 42: Temperature transmission (gateway perspective)*

The 3 red accolades used to the left of the photo shows the different output groups we can distinguish. Every section is as follow:

- The first part is about ACK requested by the sensor node and related information.
- The second part is similar to the LoRa ping pong output as explained previously. The most relevant information in this section are:
  - The control packet information sequence (^p): 1,24,8,0,11,8,-97.
  - The control radio information sequence (^r): 125,5,12.
  - The time stamp (^t): 2018-08-02T23:40:36.131.

  The 3 lines summarize information about the received packet in a compact format that can be further used later for data analysis.
- The third and last part is related to the content of the packet on itself, which is the sensed temperature 23.75 °C as received from the sensor node.

A python script in the gateway will then upload the received data to the Thing Speaks cloud. The gateway uploads to a public account set by Professor Congduc Pham as explained in his documentation for the purpose of test.

We can see on the displayed output in figure 41 that the uploaded information will be the temperature “23.750000” followed by the (^p) sequence “1,24,8,0,11,8,-97”, the (^r) sequence “125,5,12” and the (^t) sequence “2018-08-02T23:40:36+02:00”. The last sequence “00000027EB7E32F4” is the MAC address of the gateway (the raspberry pi 3) to identify which gateway is uploading to the cloud in case of multiple gateways transmission.

To see the received data, we Log in to the Web admin interface of the gateway and press the button “Get post-processing.log file” *(Button highlighted with a red arrow in figure 42)*.

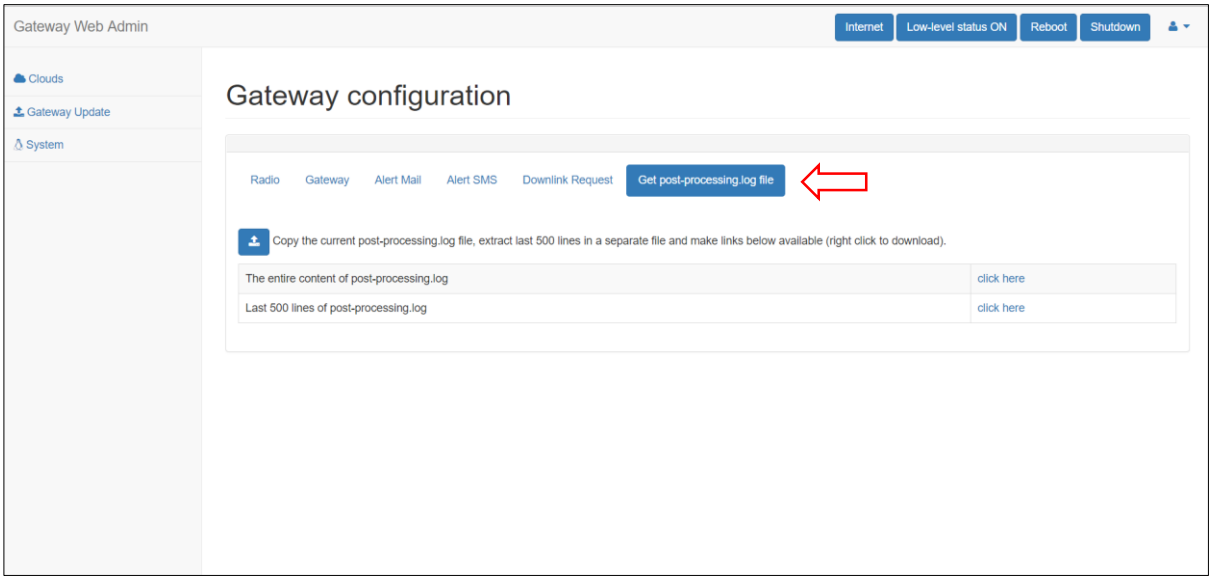



*Figure 43: the Gateway web admin interface*

We have two options to choose from them:

- Get the entire log file.
- Get the last 500 line from the log file.

This is an example of the last 500 lines from the log file (figure 43). When we scroll down to the bottom of the page we see the last transmission we tested:

Temperature measurement equal to 23.250000°C received with (^p) sequence equal to “1,24,8,0,11,9,-94”, (^r) sequence equal to “125,5,12” and time stamp (^t) equal to “2018-08-03T01:45:13+02:00”. Data received from the same gateway with ID equal to “00000027EB7E32F4”.

```
2018-08-03T01:45:13.560023> ## ACK set and written in FIFO ##
2018-08-03T01:45:13.560364> ## ACK to send:
2018-08-03T01:45:13.560562> Destination: 8
2018-08-03T01:45:13.560753> Source: 1
2018-08-03T01:45:13.560941> ACK number: 0
2018-08-03T01:45:13.561132> ACK length: 2
2018-08-03T01:45:13.561321> ACK payload: 0
2018-08-03T01:45:13.561509> ACK SNR last rcv pkt: 9
2018-08-03T01:45:13.561696>  ##
2018-08-03T01:45:13.561880>
2018-08-03T01:45:13.562067> --> CS1
2018-08-03T01:45:13.562264> --> CAD 201
2018-08-03T01:45:13.562457> OK1
2018-08-03T01:45:13.562666> --> RSSI -125
2018-08-03T01:45:13.562876> ACK requested by 8
2018-08-03T01:45:13.563084> --- rxlora. dst=1 type=0x18 src=8 seq=0 len=11 SNR=9 RSSIpkt=-94 BW=125 CR=4/5 SF=12
2018-08-03T01:45:13.563316> 2018-08-03T01:45:13.561935
2018-08-03T01:45:13.563568> rcv ctrl pkt info (^p): 1,24,8,0,11,9,-94
2018-08-03T01:45:13.563792> splitted in:  [1, 24, 8, 0, 11, 9, -94]
2018-08-03T01:45:13.564006> (dst=1 type=0x18(DATA ACK_REQ) src=8 seq=0 len=11 SNR=9 RSSI=-94)
2018-08-03T01:45:13.564293> rcv ctrl radio info (^r): 125,5,12
2018-08-03T01:45:13.564568> splitted in:  [125, 5, 12]
2018-08-03T01:45:13.564873> (BW=125 CR=5 SF=12)
2018-08-03T01:45:13.565098> rcv timestamp (^t): 2018-08-03T01:45:13.559
2018-08-03T01:45:13.565311>
2018-08-03T01:45:13.565505> got first framing byte
2018-08-03T01:45:13.565703> --> got LoRa data prefix
2018-08-03T01:45:13.565948> valid app key: accept data
2018-08-03T01:45:13.566170> number of enabled clouds is 1
2018-08-03T01:45:13.566391> --> cloud[0]
2018-08-03T01:45:13.566610> uploading with python CloudThingSpeak.py
2018-08-03T01:45:13.566821> python CloudThingSpeak.py "23.250000" "1,24,8,0,11,9,-94" "125,5,12" "2018-08-03T01:45:13+02:00" "00000027EB7E32F4"
2018-08-03T01:45:14.375382> ThingSpeak: uploading (multiple)
2018-08-03T01:45:14.375795> rcv msg to log (\!) on ThingSpeak ( default , default ):
2018-08-03T01:45:14.376007> ThingSpeak: will issue curl cmd
2018-08-03T01:45:14.376197> curl -s -k -X POST --data field1=23.250000 https://api.thingspeak.com/update?key=SGSH52UGPVAUYG3S
2018-08-03T01:45:14.376405> ThingSpeak: returned code from server is 175049
2018-08-03T01:45:14.381177> --> cloud end
2018-08-03T01:45:14.381533>
```

*Figure 44: the Log file in the Gateway*

A script that performs character reading over the log file can extract precisely the temperature value from the text.

Another way to see the data received from the sensor node is from the Thing Speak cloud dashboard logged with the public account of Professor Congduc Pham made for public tests.

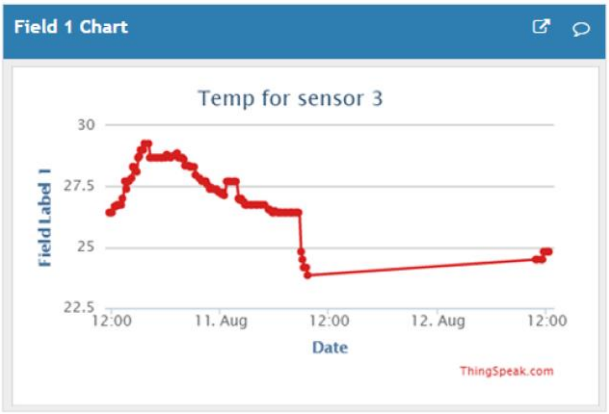



*Figure 45: temperature received as shown in The Thing Speaks Dashboard*

The same code used for temperature transmission measured with the DS18b20 sensor can be easily edited to transmit values of soil moisture (SKU:SEN0193) and values of humidity and temperature of the air (DHT22).

## 4. Connecting the whole system

After testing each module individually and making sure all of them work properly, the final step in our project is to connect all the sensors at the same board and make the raspberry pi takes simultaneously all the measurements.

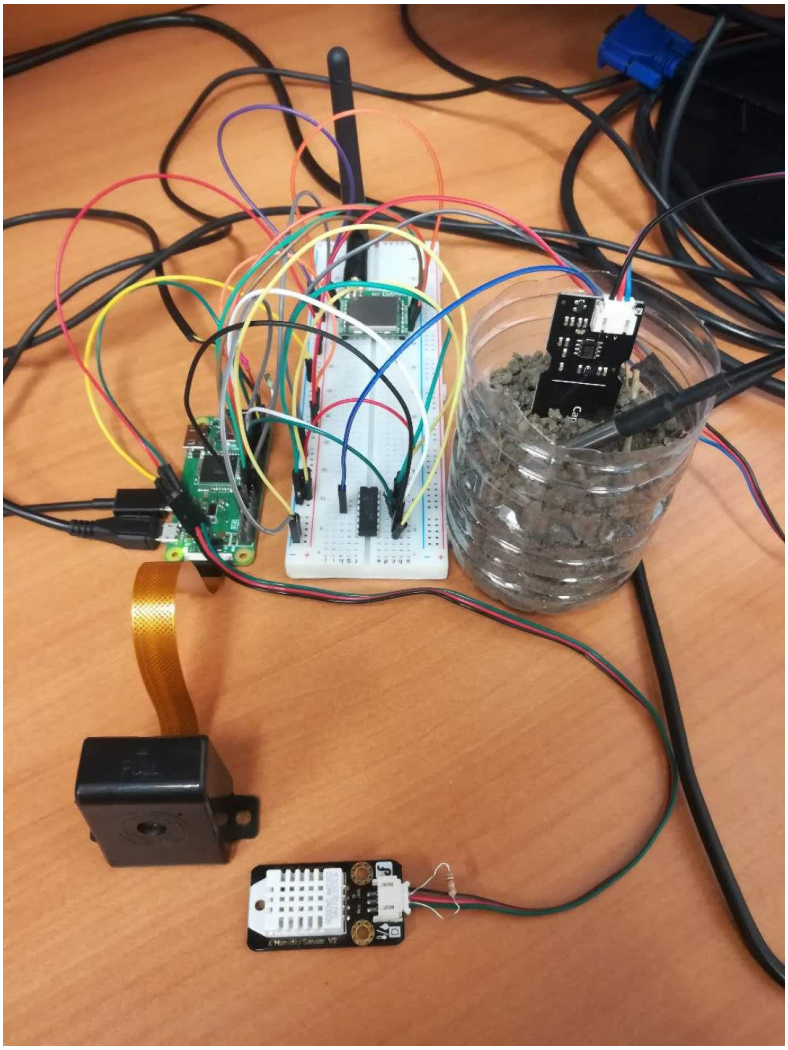


*Figure 46: The sensor node fully assembled*

Figure 45 shows the final result of how the sensor node looks like when it's fully assembled. We can see the raspberry pi zero board over it all the sensors are connected: the DS18b20 and SKU:SEN0193 sensors are put in a container with soil for real-conditions simulation. The DHT22 is kept on the air for atmospheric measurements, and the camera is connected to the CSI interface. The sensor node is connected to the Lora module and ready to send data to the gateway.

We developed a Schell script that will make all the sensors run in a cycle. The output of this script is shown in figure 46:

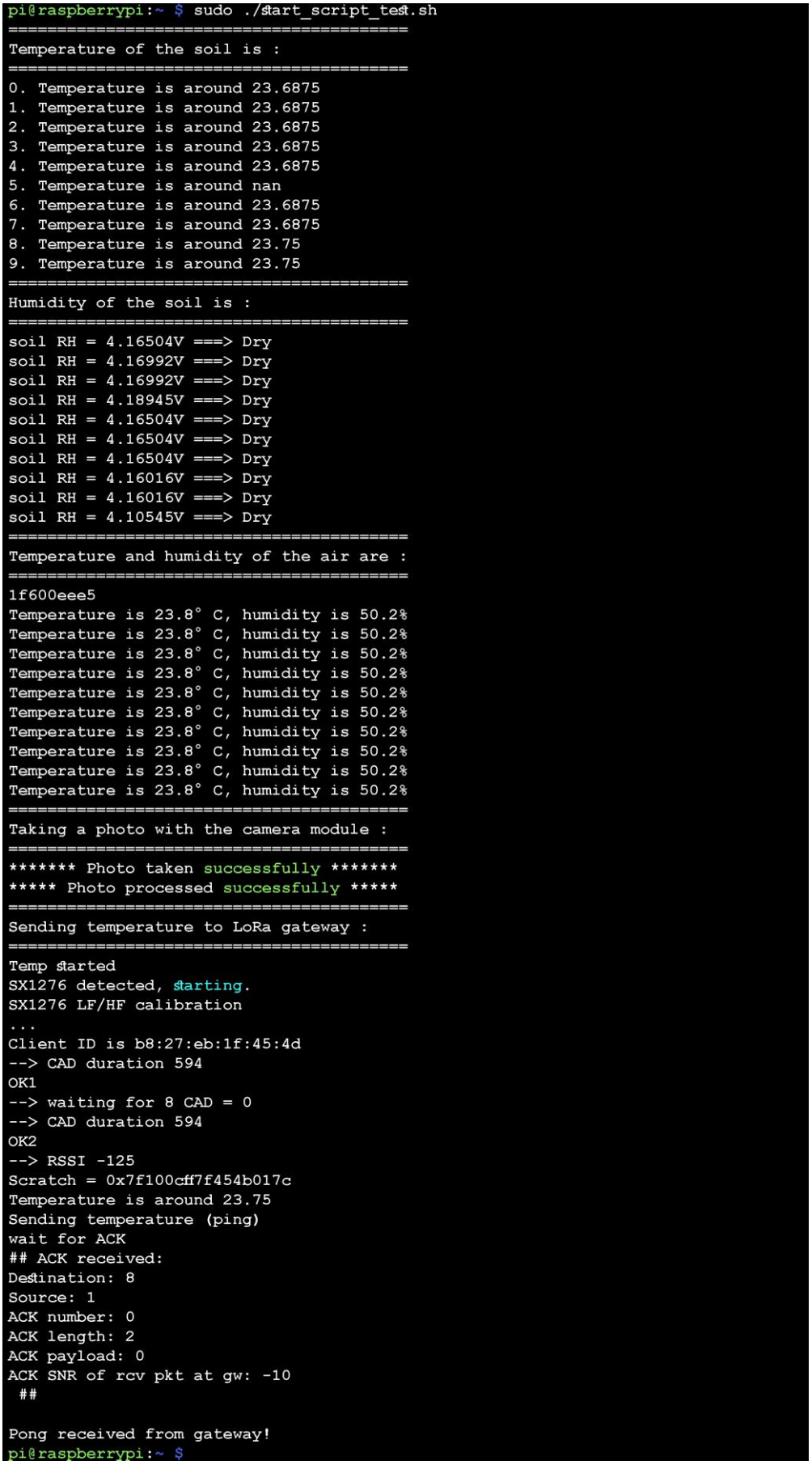



*Figure 47: Output sample of the final sensor node*

The previous script can be put inside a loop so that the sensor node take the necessary measurements frequently and send them to the gateway as how it is supposed to do in reality.

## Conclusion

In this chapter, we implemented a prototype for the sensor node and the gateway following the technical design we made in the second chapter. As a first step we tested each sensor individually, then once everything works as expected the last step was simply to assemble the whole components and run all of them with a shell script.

Our prototype is fully functional and provides the desired results.

# General conclusion

Under the constraint of climate change which affects negatively the agriculture, and knowing that important resources -mainly water- will be scarcer in the future, trying to keep up with the increasing demand of food for a super-fast growing population becomes a huge challenge for the humanity.

Fortunately, we have Technology! Smart devices connected together in an Internet of Things network will be in charge of monitoring all the required parameters for a better food quality and an optimal resources consumption, making farmer more precise and pushing agriculture to the next level: the Smart agriculture.

At ÉTS labs, we built a prototype as an application of a smart agriculture project for a client in need of a technological solution allowing him to have control over his vineyard. Our solution will monitor critical parameters in the atmosphere as well as in the soil, for a precise control over the plants, allowing our client to ensure a better fruit quality, a better health-control over his plants, and an optimal irrigation system to reduce water consumption.

To achieve our goal, we designed carefully our system. We used more adequate platforms to support easily future enhancement. We made different tests before assembling the final product. Even though our system was for a vineyard monitoring, the same concept can be used for any other crops.

Several enhancements can be added to our work as future perspectives. For instance, a private cloud or database can be developed for data storage. Actuators can be connected to our system to make all the necessary interventions as automatic as possible, reducing the frequency of human interventions. A smartphone application can be also developed for a faster access from anywhere.

# References


**[1] Mitacs web site**
https://www.mitacs.ca/en/programs/globalink/globalink-research-internship
*(Visited on Aug. 2018)*
**[2] ETS web site**
https://www.etsmtl.ca/A-propos/ETS-en-bref/Presentation
*(Visited on Aug. 2018)*
**[3] Synchromedia Lab web site**
http://www.synchromedia.ca/
*(Visited on Aug. 2018)*
**[4] [5] (Book) Interconnecting smart objects with IP: the next internet**
Written by Jean-Philippe Vasseur and Adam Dunkels, 2010
*(Visited on Mar. 2018)*
**[6] [7] Intro to Internet of Things, Marco Zennaro, PhD, December 2016**
https://www.itu.int/en/ITU-D/Regional-Presence/AsiaPacific/SiteAssets/Pages/Events/2016/Dec-2016-IoT/IoTtraining/IoT%20Intro-Zennaro.pdf
*(Visited on Apr. 2018)*
**[8] [9] [10] [11] *« Comment l'évolution actuelle d'Internet transforme-t-elle le monde ? »* by Dave Evans, April 2011**
https://www.cisco.com/c/dam/global/en_ca/solutions/executive/assets/pdf/internet-of-things-fr.pdf
*(Visited on Apr. 2018)*
**[12] International Chamber of Shipping /Shipping and world trade, 2017**
http://www.ics-shipping.org/shipping-facts/shipping-and-world-trade
*(Visited on Mai. 2018)*
**[13] [16] Smart Farming, ISO focus #122, Mai-June 2017**
https://www.iso.org/files/live/sites/isoorg/files/news/magazine/ISOfocus%20(2013-NOW)/en/2017/ISOfocus_122/ISOfocus_122_EN.pdf
*(Visited on Mai. 2018)*
**[14] UN news "World population to hit 9.8 billion by 2050 despite nearly universal lower fertility rates" - June 2017**
https://news.un.org/en/story/2017/06/560022-world-population-hit-98-billion-2050-despite-nearly-universal-lower-fertility
*(Visited on Apr. 2018)*
**[15] How to feed the world in 2050, FAO**
http://www.fao.org/fileadmin/templates/wsfs/docs/expert_paper/How_to_Feed_the_World_in_2050.pdf
*(Visited on Apr. 2018)*
**[17] World Health Organization, Drinking-water – February 2018**
http://www.who.int/en/news-room/fact-sheets/detail/drinking-water
*(Visited on Mai. 2018)*
**[18] Ventilation for designing better buildings – September 2014**
http://info.zehnderamerica.com/blog/absolute-vs.-relative-humidity-whats-the-difference
*(Visited on Mai. 2018)*
**[19] Illustration source**
https://fr.wikipedia.org/wiki/%C3%89vapotranspiration
*(Visited on Aug. 2018)*
**[20] [21] *« Rapport de conception ELE795 » Summer 2017***
By Philippe Chabot, Vincent Grosselin and Xavier Mercure-Gagnon *(no link available)*
*(Read on Mar. 2018)*
**[22] Arduino web site**
https://www.arduino.cc/en/Tutorial/Memory
*(Visited on Aug. 2018)*

**[23] North Star Imaging "Size for a digital image"**
https://4nsi.com/faq/how-do-i-calculate-the-file-size-for-a-digital-image
*(Visited on Jun. 2018)*
**[24] Single board computer benchmark, by Shawnhymel**
https://learn.sparkfun.com/tutorials/single-board-computer-benchmarks
*(Visited on Apr. 2018)*
**[25] IGEP v2 - datasheet**
https://www.isee.biz/support/downloads/item/igepv2-datasheet
*(Visited on Apr. 2018)*
**[26] Raspberry pi zero – power consumption comparison, by Jeff Geerling – Novembre 2015**
https://www.jeffgeerling.com/blogs/jeff-geerling/raspberry-pi-zero-power
*(Visited on Apr. 2018)*
**[27] Raspberry pi zero OS list**
https://www.kodifiretvstick.com/raspberry-pi-zero-operating-system-list/
*(Visited on Mai. 2018)*
**[28] Raspberry pi web site / Download page**
https://www.raspberrypi.org/downloads/
*(Visited on Mai. 2018)*
**[29] *« Comparatif des modèles de Raspberry pi »* by Social compare, August 2018**
http://socialcompare.com/fr/comparison/raspberrypi-models-comparison
*(Visited on Apr. 2018)*
**[30] 802.15.4 wireless for IoT developers by Mike Vidales – Helium Blog, May 2017**
https://blog.helium.com/802-15-4-wireless-for-internet-of-things-developers-1948fc313b2e
*(Visited on Jul. 2018)*
**[31] "11 IoT protocols you need to know" April 2015**
https://www.rs-online.com/designspark/eleven-internet-of-things-iot-protocols-you-need-to-know-about
*(Visited on Jul. 2018)*
**[32] "Is Lora the game-changer for IoT?" April 2016**
https://internetofbusiness.com/lora-game-changer-iot/
*(Visited on Apr. 2018)*
**[33] Illustration source**
https://fr.pinout.xyz/pinout/wiringpi
*(Visited on Mai. 2018)*
**[34] Image classification with HSV Color Model Processing, by Jeefri A. Moka – October 2017**
https://www.datasciencecentral.com/profiles/blogs/image-classification-with-hsv-color-model-processing
*(Visited on Jun. 2018)*
**[35] Low-cost LoRa IoT framework, by C. Pham**
https://github.com/CongducPham/LowCostLoRaGw#connect-a-radio-module-to-your-end-device
*(Visited on Jul. 2018)*
**[36] A DIY low-cost LoRa gateway, by C. Pham**
http://cpham.perso.univ-pau.fr/LORA/RPIgateway.html
*(Visited on Jul. 2018)*
**[37] Reading channel RSSI during a CAD, by SEMTECH**
https://www.semtech.com/uploads/documents/an1200.21_std.pdf
*(Visited on Jul. 2018)*

# ANNEXE

# Conception of an IoT network for vineyard monitoring: Instructions' Guide

***(Mitacs internship, summer 2018)***
***By Taha BEN SALAH***

## The sensor node:

The raspberry pi zero card represents the sensor node.

- Login: pi
- Password: ele795

Once logged in, you will be in the /home/pi location. You will see 3 folders and one executable file.

### 1- Image (folder)

It contains files related to the Raspberry Pi camera module: the "LeavesDetection" code is responsible for applying filters over an input image to make a black and white output image. To test this code use the command:

sudo ./LeavesDetection [input] ; with [input] refers to your input image.

You can use for test your own photo shot by the camera by running the python code "photo_shooting.py" or simply use the provided sample photos in the "test_images" folder. The "results" folder contains the output image you will have once "LeavesDetection" is run on the sample images from "test_images".

### 2- Single-sensor-tests (folder)

It contains different codes related to each individual sensor for test purpose:

- The "adc" folder is for the analogue to digital converter MCP 3008.
- The "common" folder contains files required by various codes of each sensor, like the *hwconfig* file for pins declaration, or the *arduPi* (a C++library which allows you developing codes for Raspberry as if it were for Arduino) etc.
- The "dht22" folder contains code for the combined sensor temperature and humidity of the air.
- DS18b20 is the soil temperature sensor.
- Lora folder contains code for LoRa ping test.
- Soil_rh folder contains code for the soil humidity SKU: SEN0193 sensor.
- As its name suggests, the executable_copy folder contains copies as backup files in case of problems.

### 3- Temp (folder)

It contains code for data transmission through the LoRa module. The data transmitted through this code are the temperature measured by the DS18b20 sensor (which justify the name Temp, a short form of temperature).

The same code can be modified and adapted for sending other data sensed by the rest of the sensors.

### 4- Start_script_test.sh

This executable file is a script Schell written in order to make all the sensors function at the same time. This file was written to simulate an application of our sensor node and how it is supposed to run all the sensors at the same time to collect various measures.

A loop is needed to adjust the frequency of measurement required, and then save it as an automatic file to be run immediately after boot up. This will make our sensor node fully autonomous: once put in the farm and plugged into an energy source, it will start automatically taking all the measure frequently.

## The Gateway

The raspberry pi 3 model B is our LoRa Gateway.

- Login: pi
- Password: loragateway

The OS flashed on its SD card is as described by Prof Congduc Pham's documentation:

*https://github.com/CongducPham/LowCostLoRaGw#connect-a-radio-module-to-your-end-device*

*(Further information can be found on his documentation about setting up and installation)*

Once logged in the gateway, you will be in a graphical view. You can switch to tty1 or tty2 environments by pressing respectively Ctrl+Alt+F1 and Ctrl+Alt+F2. To switch back to the graphical view simply press Ctrl+Alt+F7.

If you are in the graphical view and you run from the sensor node the Temp code or the LoRa ping_pong code, you will not see any messages appear on the terminal about exchanges between node and gateway. For this purpose simply switch to the tty1 or tty2 environments and you will see messages informing you about data received.

*(PS: this applies only to the Gateway, for the sensor node you will see always the exchanged messages on its terminal).*

## Circuit diagrams

Generated by *Fritzing* software.

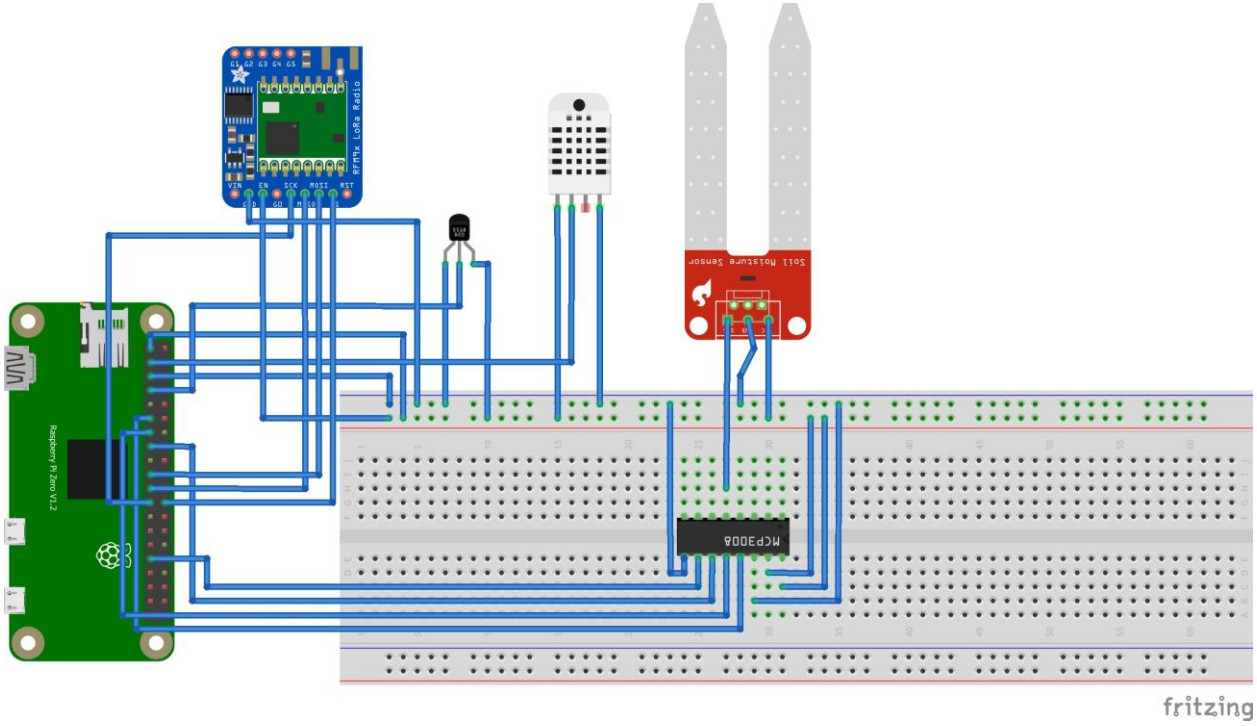



*Figure 48: Breadboard view of the sensor node*

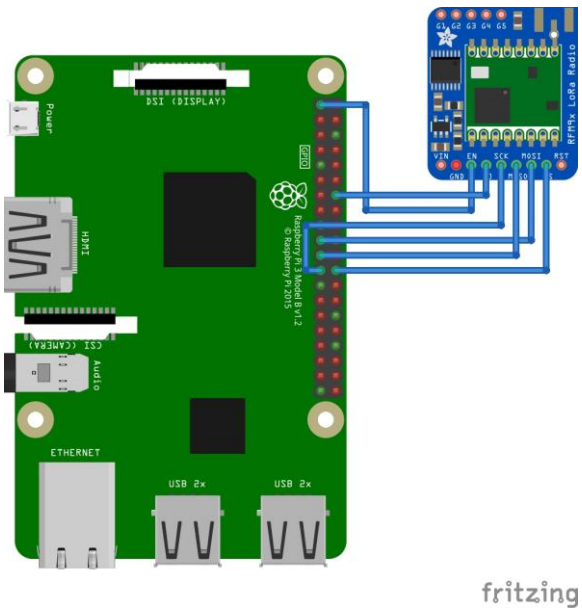



*Figure 49: Breadboard view of the gateway*

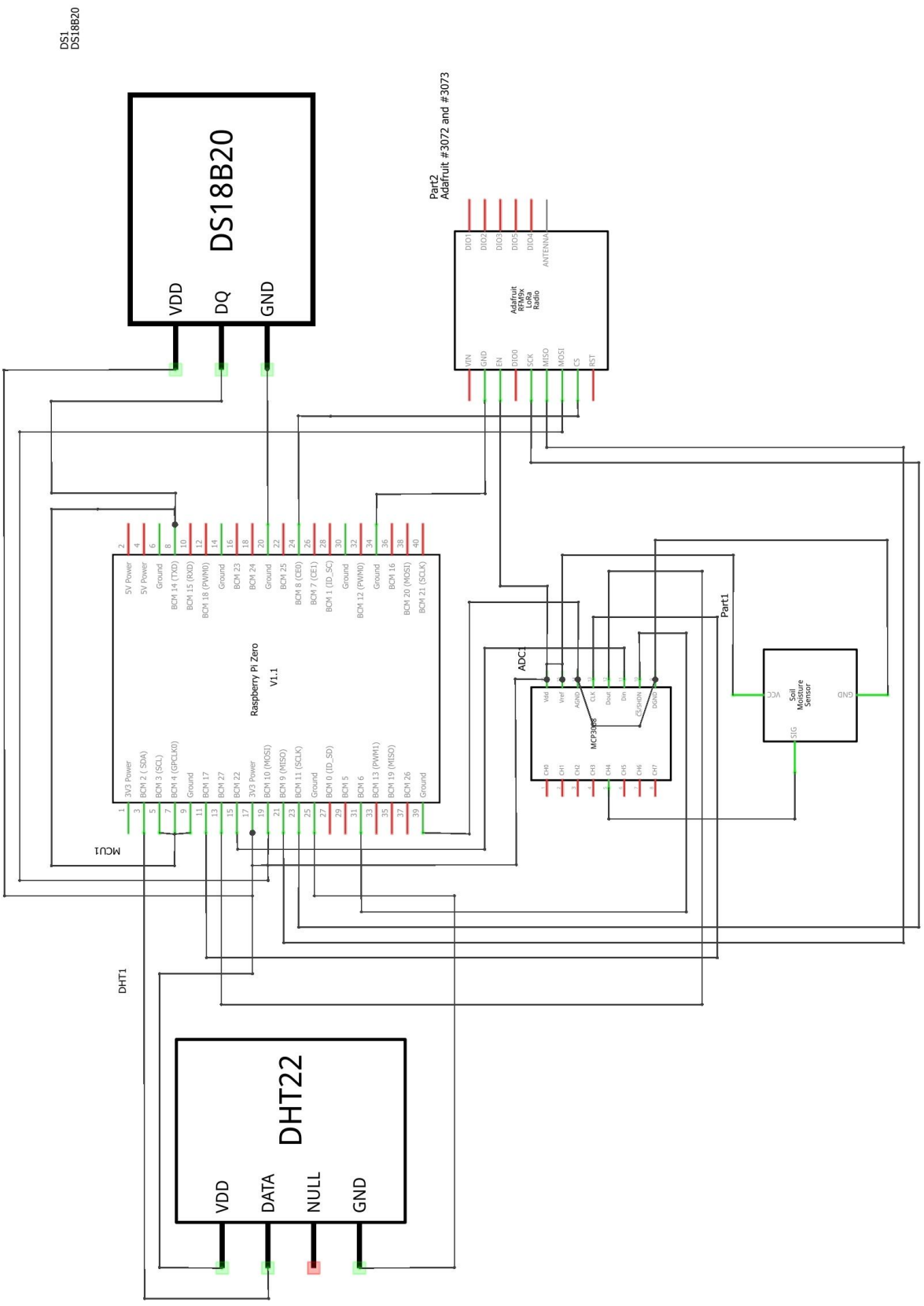



*Figure 50: Schematic view of the sensor node*

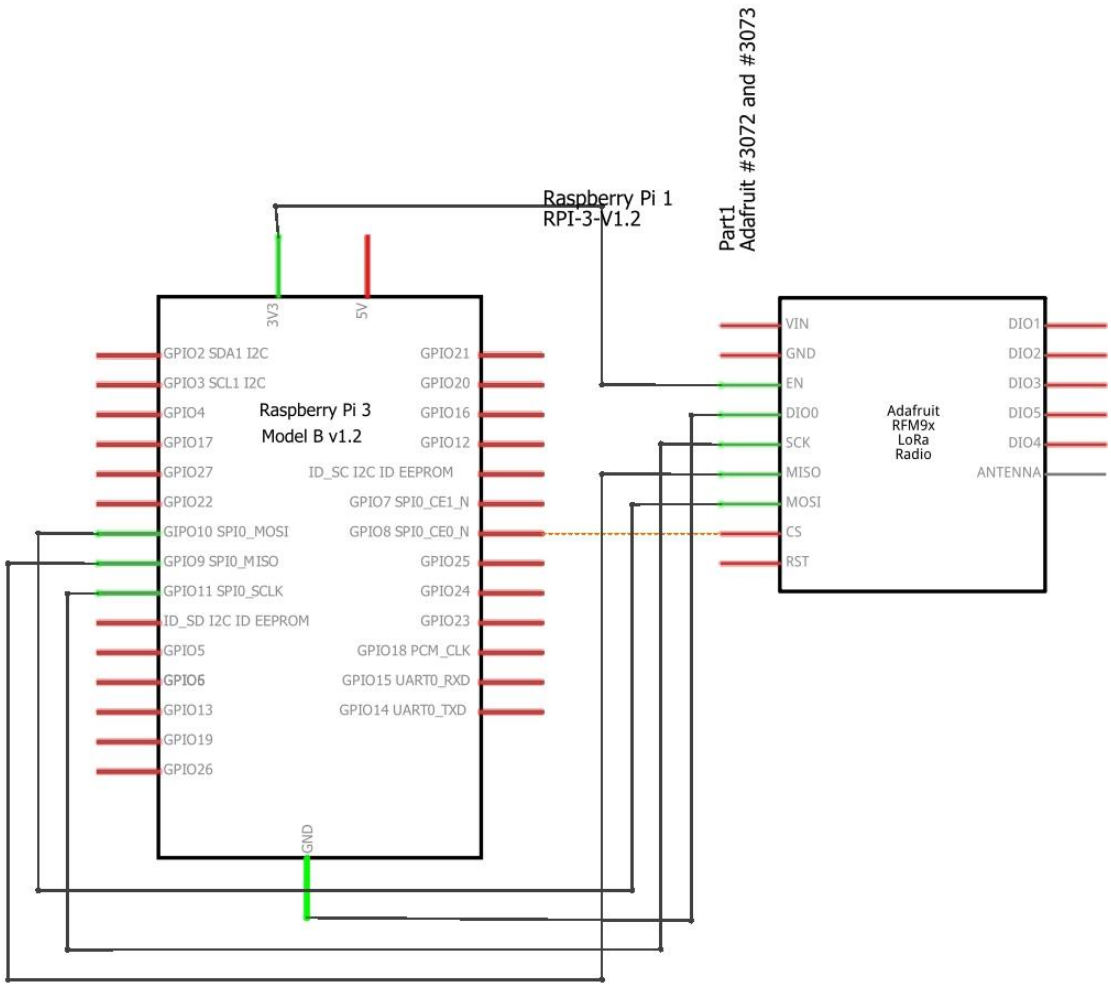



*Figure 51: Schematic view of the Gateway*